



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de **HONORIS UNITED UNIVERSITIES**

Rapport de Projet de Fin d'Année

4ème année — Filière 4DSIO G3

Conception et Développement d'une Plateforme de Gestion de Produits, Commandes et Facturation

ProduitFlow

Réalisé par :

- Imad EL GABBAS
- Manal LAHMER
- Aymen NAFIS

Encadré par :

- Mme. ZINEB BENYAHYA
- Mme. BOUSMAR KHADIJA
- Mme. NAJI ZINEB

Année universitaire : 2025/2026

Dédicaces

Nous dédions ce travail à nos familles, dont le soutien inconditionnel, la confiance et les encouragements ont été notre principale source de motivation tout au long de ce projet. Votre présence à chaque étape de notre parcours académique a été une source d'énergie et de persévérance inestimable.

À nos encadrants, pour leur disponibilité, leurs précieux conseils et leur accompagnement rigoureux. Leur expertise respective en développement front-end, en entrepreneuriat et en gestion de projet a profondément enrichi notre approche et la qualité de notre travail.

À tous nos enseignants de l'École Marocaine des Sciences de l'Ingénieur, dont les enseignements ont forgé nos compétences techniques et humaines au fil de ces quatre années de formation.

À nos camarades de promotion du groupe G3, pour les échanges constructifs, le soutien mutuel et les moments de collaboration qui ont rendu ce parcours académique plus enrichissant.

Enfin, à toutes les personnes qui, de près ou de loin, ont contribué à l'aboutissement de ce travail.

Remerciements

Au terme de ce travail, nous tenons à exprimer notre sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'année.

Nous remercions chaleureusement nos encadrants pour leur suivi attentif, leurs orientations précieuses et le temps qu'ils ont consacré à l'encadrement de ce projet tout au long du semestre. Leurs expertises respectives en développement front-end React, en gestion de projet et en entrepreneuriat ont été déterminantes dans l'aboutissement de ce travail.

Nous adressons également notre gratitude à l'ensemble du corps professoral et administratif de l'EMSI pour la qualité de la formation dispensée et pour l'environnement académique stimulant qu'ils ont su créer.

Nos remerciements vont aussi aux membres du jury qui nous font l'honneur d'évaluer ce travail et d'apporter leur regard critique et constructif.

Enfin, nous remercions Dieu Tout-Puissant de nous avoir accordé la force, la patience et la persévérance nécessaires à l'accomplissement de ce projet.

Résumé

Ce rapport présente la conception et le développement de ProduitFlow, une plateforme web full-stack de gestion de produits, commandes et facturation, réalisée dans le cadre du Projet de Fin d'Année de la filière 4DSIO (Développement des Systèmes d'Information et des Objets connectés) à l'École Marocaine des Sciences de l'Ingénieur, année universitaire 2025/2026.

La solution propose deux espaces distincts et complémentaires : un espace administrateur permettant la gestion complète des produits (CRUD), des commandes (approbation/refus avec décrémentation automatique du stock), des factures (générées automatiquement à l'approbation) et des utilisateurs (blocage, suppression) ; et un espace utilisateur offrant un catalogue de produits avec filtres avancés, un système de panier basé sur Zustand, et un suivi de commandes et factures personnalisé.

La stack technique repose sur React 18 avec TypeScript et Vite côté frontend, Node.js avec Express et TypeScript côté backend, MongoDB comme base de données NoSQL, Zustand pour la gestion de l'état global du panier, et JSON Web Token pour l'authentification sécurisée. Le rapport couvre l'intégralité du cycle de développement : analyse des besoins, conception UML, gestion de projet (PERT, Gantt, workflow Git), réalisation technique et étude entrepreneuriale complète incluant analyse de marché, marketing stratégique et faisabilité financière.

Mots-clés : React 18, TypeScript, Node.js, Express, MongoDB, Mongoose, Zustand, JWT, bcryptjs, Gestion de produits, Commandes, Facturation automatique, SaaS, PME, SARL.

Abstract

This report presents the design and development of ProduitFlow, a full-stack web platform for product, order, and invoice management, developed as a final year project for the 4DSIO program at the École Marocaine des Sciences de l'Ingénieur, academic year 2025/2026.

The solution provides two distinct spaces: an administrator dashboard for complete management of products (CRUD), orders (approval/rejection with automatic stock decrement), invoices (auto-generated upon order approval), and users; and a user space featuring a product catalog with advanced filters, a Zustand-based cart system, and personalized order and invoice tracking.

The technical stack includes React 18 with TypeScript and Vite on the frontend, Node.js with Express and TypeScript on the backend, MongoDB as the NoSQL database, Zustand for global cart state management, and JSON Web Tokens for secure authentication.

Keywords: React 18, TypeScript, Node.js, Express, MongoDB, Mongoose, Zustand, JWT, Product Management, Orders, Automatic Invoicing, SaaS, SME.

Table des Matières

Dédicaces	2
Remerciements.....	3
Résumé	4
Abstract	5
Table des Matières.....	6
Liste des Figures	9
Liste des Tableaux.....	10
Introduction Générale	11
Chapitre 1 : Contexte Général du Projet	12
1. Introduction	12
2. Présentation du Projet	12
2.1. Étude de l'existant	12
2.2. Critique de l'existant	12
2.3. Solution Proposée	13
3. Méthodologie de Développement	14
4. Conclusion	15
Chapitre 2 : Gestion de Projet	16
1. Introduction	16
2. Organisation de l'Équipe et Répartition des Rôles	16
3. Planification des Tâches	17
4. Réseau PERT	18
5. Diagramme de Gantt	19
6. Workflow Git	20
7. Conclusion	21
Chapitre 3 : Spécification des Besoins.....	22
1. Introduction	22
2. Identification des Acteurs	22
3. Besoins Fonctionnels	22
3.1. Module Authentification	22
3.3. Module Catalogue (User)	23
3.4. Module Fiche Produit et Panier	23
3.5. Module Commandes.....	24
3.6. Module Factures	24
3.7. Module Utilisateurs (Admin)	25
3.8. Tableau de Bord Admin	25
4. Besoins Non Fonctionnels	25

4.1. Performance	25
4.2. Sécurité.....	26
4.3. Maintenabilité.....	26
4.4. Ergonomie et Accessibilité	26
5. Diagrammes de Cas d'Utilisation.....	26
6. Conclusion	29
Chapitre 4 : Conception du Système	30
1. Introduction	30
2. Modélisation Dynamique	30
2.1. Diagrammes de Séquences	30
2.2. Diagramme d'Activité.....	33
3. Modélisation Statique	34
3.1. Diagramme de Classes.....	34
3.2. Types TypeScript Partagés	36
3.3. Architecture de l'Application	36
3.4. Structure du Frontend	38
3.5. Routes Backend Complètes	38
4. Conclusion	40
Chapitre 5 : Réalisation du Système.....	41
1. Introduction	41
2. Environnement de Développement	41
2.1. Environnement Matériel.....	41
2.2. Stack Technique Complète	41
3. Variables d'Environnement Backend.....	42
4. Interfaces Graphiques Principales.....	45
4.1. Authentification	45
4.2. Espace Administrateur — AdminLayout.....	46
4.3. Espace Utilisateur — UserLayout.....	49
5. Points Techniques Clés Implémentés	51
5.1. Architecture Auth avec AuthContext et useReducer	51
5.2. Gestion du Panier avec Zustand	52
5.3. Populate Mongoose et Helper getUserNom()	52
5.4. Workflow Commande → Stock → Facture	52
5.5. Corrections d'Intégration	52
6. Conclusion	53
Chapitre 6 : Étude Entrepreneuriale	54
1. Introduction	54
2. Contexte et Identification du Problème.....	54
3. Présentation de la Solution et Valeur Ajoutée.....	54

4. Origine de l'Idée Entrepreneuriale	54
Business Model Canvas	55
5. Étude de Marché	56
5.1. Analyse de la Demande.....	56
5.2. Analyse de la Concurrence	56
5.3. Analyse des Fournisseurs	57
5.4. Réglementations du Secteur.....	57
5.5. Outils d'Analyse Stratégique.....	57
6. Marketing Stratégique.....	61
6.1. Segmentation, Ciblage, Positionnement (SCP).....	61
6.2. Marketing Mix (4P).....	62
6.3. Stratégie d'Acquisition Clients	62
6.4. Stratégie de Confiance	63
7. Étude de Faisabilité Technique.....	63
8. Étude de Faisabilité Financière.....	64
8.1. Investissement Initial	64
8.2. Prévisions de Ventes — Scénario Conservateur (Année 1)	64
8.3. Charges Variables	65
8.4. Charges Fixes Mensuelles	65
8.5. Besoin en Fonds de Roulement (BFR)	65
8.6. Trésorerie Prévisionnelle — 12 Premiers Mois	66
8.7. Résultat Prévisionnel et Seuil de Rentabilité	67
9. Aspect Juridique.....	67
10. Stratégie de Développement.....	68
11. Conclusion.....	68
Conclusion Générale et Perspectives	69
Bibliographie et Webographie.....	71
Documentation Officielle.....	71
Ouvrages de Référence	71
Ressources en Ligne.....	71

Liste des Figures

Figure 1 : Architecture générale de ProduitFlow.....	14
Figure 2 : Réseau PERT du projet	18
Figure 3 : Diagramme de Gantt du projet	20
Figure 4 : Diagramme de cas d'utilisation — Espace Administrateur.....	28
Figure 5 : Diagramme de cas d'utilisation — Espace Utilisateur.....	28
Figure 6 : Diagramme de séquence — Authentification (Login).....	31
Figure 7 : Diagramme de séquence — Passer une commande	32
Figure 8 : Diagramme de séquence — Approuver une commande (Admin).....	33
Figure 9 : Diagramme d'activité — Cycle de vie complet d'une commande.....	34
Figure 10 : Diagramme de classes — ProduitFlow.....	35
Figure 11 : : Architecture logique — Diagramme de composants.....	37
Figure 12 : Architecture matérielle — Diagramme de déploiement.....	37
Figure 13 : Logo React.....	42
Figure 14 : Logo Zustand	43
Figure 15 : Logo Vite.....	43
Figure 16 : Logo TypeScript.....	43
Figure 17 : Logo AXIOS	43
Figure 18 : Logo node.js	44
Figure 19 : Logo mongoose	44
Figure 20 : Logo mongoDB	44
Figure 21 : Logo JWT.....	44
Figure 22 : Logo express	44
Figure 23 : Logo Visual Studio Code.....	45
Figure 24 : Logo MongoDB Compass	45
Figure 25 : Logo GitHub.....	45
Figure 26 : Page de connexion / inscription.....	46
Figure 27 : Dashboard Administrateur — indicateurs clés.....	47
Figure 28 : Page Produits Admin — CRUD complet avec modal	47
Figure 29 : Page Commandes Administrateur — approbation et filtrage	48
Figure 30 : Page Factures Administrateur	48
Figure 31 : : Page Utilisateurs Administrateur — blocage et suppression.....	49
Figure 32 : Catalogue Produits — grille avec filtres avancés.....	49
Figure 33 : Fiche Produit — sélecteur de quantité borné au stock.....	50
Figure 34 : Page Panier — gestion des articles et passage de commande	50
Figure 35 : Page Commandes Utilisateur — statut coloré et modal détail	51
Figure 36 : Page Factures Utilisateur — liste et modal détail	51
Figure 37 : Matrice SWOT de ProduitFlow	58
Figure 38 : Analyse PESTEL de l'environnement de ProduitFlow	59
Figure 39 : Analyse des 5 Forces de Porter — ProduitFlow	60

Liste des Tableaux

Tableau 1 : Répartition des tâches par membre	16
Tableau 2 : Tableau des tâches avec durées et dépendances	17
Tableau 3 : Calcul des marges et chemin critique	19
Tableau 4 : Récapitulatif des besoins fonctionnels par module	25
Tableau 5 : Description des cas d'utilisation principaux.....	27
Tableau 6 : Types partagés — src/types/index.ts	36
Tableau 7 : Structure du frontend — organisation des dossiers src/.....	38
Tableau 8 : Routes Backend RESTful — récapitulatif complet	38
Tableau 9 : Stack technique — environnement de développement	41
Tableau 10 : Business Model Canvas (BMC)	55
Tableau 11 : Analyse de la concurrence	56
Tableau 12 : Analyse des fournisseurs	57
Tableau 13 : Tableau de la Matrice SWOT	58
Tableau 14 : Tableau d'Analyse PESTEL	59
Tableau 15 : Tableau des 5 Forces de Porter	61
Tableau 16 : Ressources techniques et humaines requises.....	63
Tableau 17 : Investissement initial	64
Tableau 18 : Prévisions de ventes (scénario conservateur)	64
Tableau 19 : Charges variables	65
Tableau 20 : Charges fixes mensuelles	65
Tableau 21 : Calcul du Besoin en Fonds de Roulement (BFR)	66
Tableau 22 : Trésorerie prévisionnelle — 12 premiers mois.....	66
Tableau 23 : Résultat prévisionnel et seuil de rentabilité.....	67

Introduction Générale

La transformation numérique des entreprises constitue aujourd'hui l'un des enjeux majeurs de l'économie mondiale. Dans ce contexte en perpétuelle évolution, les petites et moyennes entreprises (PME) sont confrontées à un défi de taille : digitaliser leurs processus internes de gestion tout en maîtrisant leurs coûts et en préservant leur agilité opérationnelle. Au Maroc, cette réalité est d'autant plus prégnante que le tissu économique est composé à plus de 95% de TPE et PME, dont la grande majorité n'a pas encore entamé sa transformation digitale.

C'est dans cette perspective que s'inscrit ProduitFlow, notre projet de fin d'année réalisé dans le cadre de la filière 4DSIO (Développement des Systèmes d'Information et des Objets connectés) de l'École Marocaine des Sciences de l'Ingénieur. ProduitFlow est une plateforme web full-stack de gestion de produits, commandes et facturation, conçue pour répondre aux besoins concrets des PME souhaitant structurer et automatiser leur chaîne commerciale sans recourir à des solutions ERP coûteuses et complexes.

La solution permet à un administrateur de gérer l'ensemble du cycle commercial — produits, commandes, factures, utilisateurs — tandis que les utilisateurs finals bénéficient d'une expérience fluide : consultation d'un catalogue interactif, ajout au panier avec gestion des quantités, passage de commande et suivi en temps réel de la facturation. L'automatisation constitue le fil conducteur de l'application : la décrémentation du stock et la génération de la facture s'effectuent automatiquement à l'approbation d'une commande, sans intervention manuelle supplémentaire.

L'application repose sur une stack technique moderne et cohérente : React 18 avec TypeScript côté frontend, Node.js / Express côté backend, MongoDB comme base de données NoSQL, et Zustand pour la gestion de l'état global. Le développement a été conduit en équipe de trois membres, avec un workflow Git structuré en branches feature et des intégrations contrôlées vers la branche principale.

Ce rapport documente l'intégralité du processus de développement selon une progression logique en six chapitres complémentaires. Le premier chapitre présente le contexte général, l'étude de l'existant et la solution proposée. Le deuxième chapitre détaille la gestion de projet avec le réseau PERT, le diagramme de Gantt et le workflow Git. Le troisième chapitre formalise les besoins fonctionnels et non fonctionnels. Le quatrième chapitre expose la conception du système à travers les diagrammes UML. Le cinquième chapitre décrit la réalisation concrète. Enfin, le sixième chapitre constitue l'étude entrepreneuriale complète.

Chapitre 1 : Contexte Général du Projet

1. Introduction

Ce premier chapitre a pour objectif d'établir le cadre conceptuel et contextuel dans lequel s'inscrit ProduitFlow. Il présente le périmètre du projet, analyse les solutions existantes sur le marché, identifie leurs limites et justifie la pertinence de notre approche. Cette phase préliminaire est indispensable pour ancrer le projet dans une réalité opérationnelle concrète et démontrer la valeur ajoutée de la solution proposée.

2. Présentation du Projet

2.1. Étude de l'existant

La gestion commerciale des PME repose encore largement sur des outils fragmentés et peu intégrés. L'analyse du marché révèle trois grandes catégories de solutions utilisées par les entreprises de petite et moyenne taille :

Les tableurs génériques (Microsoft Excel, Google Sheets) constituent la ressource la plus répandue dans les PME marocaines. Accessibles et peu coûteux, ils permettent une gestion basique des stocks et des commandes mais présentent des limitations majeures : absence de gestion des rôles et des accès, aucune automatisation des processus, risques élevés d'erreurs humaines lors de la saisie, et impossibilité de gérer des flux de commandes en temps réel avec plusieurs utilisateurs simultanés.

Les ERP généralistes (Odoo, SAP Business One, Microsoft Dynamics) offrent des fonctionnalités très avancées couvrant l'ensemble des processus d'entreprise. Cependant, leur adoption par les PME se heurte à plusieurs obstacles : coût de licence élevé (plusieurs milliers de dirhams par mois), complexité d'implémentation nécessitant des consultants spécialisés, délais de déploiement longs (plusieurs mois), et courbe d'apprentissage importante pour les équipes non techniques.

Les solutions SaaS verticales (WooCommerce, Shopify, PrestaShop) sont principalement orientées e-commerce grand public (B2C) et ne répondent pas aux besoins spécifiques de la gestion interne B2B des PME. Elles ne proposent pas de workflow d'approbation des commandes par un administrateur, ni de gestion fine des rôles utilisateurs dans un contexte d'entreprise.

2.2. Critique de l'existant

L'analyse comparative des solutions disponibles révèle plusieurs lacunes communes particulièrement pénalisantes pour les PME :

- Absence de gestion intégrée des rôles administrateur / utilisateur adaptée au contexte PME : les ERP proposent des droits trop granulaires et complexes, les tableurs aucune gestion des accès.
- Manque de transparence dans le suivi du cycle de vie d'une commande : les PME ne disposent pas d'une vue unifiée de l'état de chaque commande (en attente → approuvée → facturée).
- Absence d'automatisation de la facturation : la génération manuelle des factures est chronophage et source d'erreurs.
- Décalage entre la validation d'une commande et la mise à jour du stock : les erreurs d'inventaire sont fréquentes avec les outils manuels.
- Coût prohibitif des licences ERP pour les structures de moins de 50 employés.
- Complexité d'utilisation des ERP pour des équipes commerciales sans formation technique.
- Absence de solution locale adaptée au contexte et aux pratiques des PME marocaines.

2.3. Solution Proposée

Face à ces lacunes identifiées, ProduitFlow se positionne comme une solution web full-stack légère, moderne et accessible, offrant le juste milieu entre la simplicité des tableurs et la puissance des ERP. La plateforme a été conçue selon le principe du 'just enough' : fournir exactement les fonctionnalités dont une PME a besoin pour gérer sa chaîne commerciale, sans la complexité superflue des grandes solutions du marché.

ProduitFlow intègre nativement les fonctionnalités suivantes :

- Gestion des produits avec CRUD complet, suivi du stock en temps réel, gestion par catégorie et alerte de stock faible (< 5 unités) sur le tableau de bord.
- Catalogue produits pour les utilisateurs avec grille de cards, filtres avancés (recherche par nom, filtre par catégorie, plage de prix min/max) et skeleton loading pour une expérience fluide.
- Panier d'achat basé sur Zustand avec persistance de session, sélecteur de quantité borné au stock disponible et gestion des totaux en temps réel.

- Workflow de commande complet : création par l'utilisateur, approbation ou refus par l'admin, décrémentation automatique du stock à l'approbation, et création automatique de la facture correspondante.
- Facturation automatique : toute commande approuvée génère instantanément une facture associée, consultable par l'utilisateur et l'administrateur.
- Tableau de bord administrateur avec indicateurs clés : nombre de commandes totales, revenus calculés sur les commandes approuvées uniquement, produits en stock faible, nombre d'utilisateurs actifs.
- Système d'authentification JWT avec gestion des rôles (admin / user), protection des routes côté frontend (PrivateRoute) et côté backend (middleware auth), et persistance de session via sessionStorage.

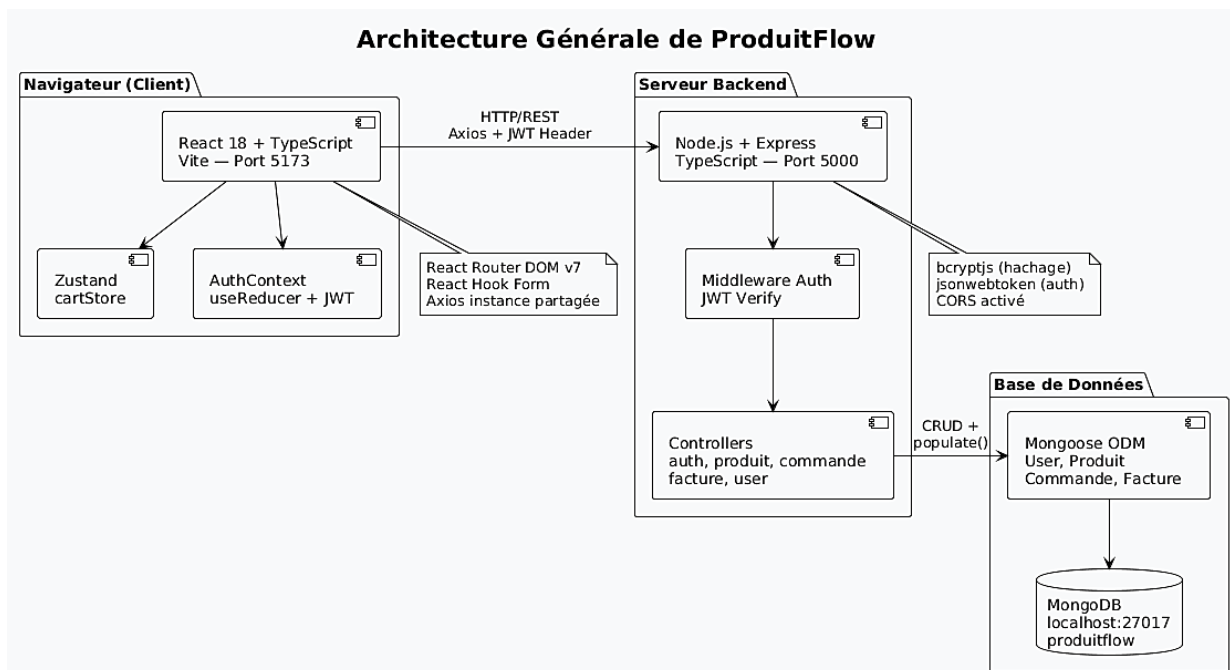


Figure 1 : Architecture générale de ProduitFlow

3. Méthodologie de Développement

Le projet a été conduit selon une approche itérative et incrémentale inspirée des principes Agile, adaptée aux contraintes d'un projet académique de groupe. Cette méthodologie hybride combine une phase de conception initiale structurée (analyse des besoins, modélisation UML, architecture technique) avec des cycles de développement courts et parallèles permettant une intégration progressive des fonctionnalités.

Chaque membre de l'équipe a travaillé sur une branche Git dédiée (feature branch), permettant un développement parallèle sans blocage mutuel. Les intégrations vers la branche principale ont été gérées exclusivement par Aymen NAFIS, responsable de l'intégration, via des Pull Requests validées sur GitHub. Cette discipline de workflow a permis de maintenir une branche main toujours stable et fonctionnelle, tout en facilitant la détection et la résolution des conflits lors des merges.

La communication au sein de l'équipe a reposé sur des réunions régulières de synchronisation permettant de valider les avancées, d'identifier les dépendances entre les modules et d'ajuster la répartition des tâches en fonction des difficultés rencontrées.

4. Conclusion

Ce premier chapitre a permis d'établir le cadre général du projet ProduitFlow, de démontrer sa pertinence face aux limites des solutions existantes et de présenter l'approche méthodologique adoptée. L'analyse de l'existant a mis en évidence un réel besoin de marché pour une solution de gestion commerciale légère et accessible, que ProduitFlow ambitionne de combler. Les chapitres suivants détailleront les aspects de planification, de conception et de réalisation technique qui ont permis de concrétiser cette vision.

Chapitre 2 : Gestion de Projet

1. Introduction

Ce chapitre présente l'organisation adoptée pour mener à bien le développement de ProduitFlow dans les délais impartis. Une gestion de projet rigoureuse est indispensable dans un contexte de développement collaboratif impliquant plusieurs membres aux compétences complémentaires. La planification a reposé sur une répartition claire des responsabilités, une visualisation des dépendances entre tâches via le réseau PERT, une projection temporelle via le diagramme de Gantt, et un workflow Git structuré garantissant la cohérence et la qualité du code intégré.

2. Organisation de l'Équipe et Répartition des Rôles

L'équipe est composée de trois membres aux responsabilités complémentaires et clairement définies dès le lancement du projet :

Tableau 1 : Répartition des tâches par membre

Membre	Rôle	Responsabilités principales	Branche Git
Aymen NAFIS	Lead Intégration (Membre 1)	Setup projet, backend complet (Express, MongoDB, Auth JWT), gestion produits/utilisateurs, intégration branches, tests et corrections	main + feature/auth- produits-utilisateurs
Imad EL GABBAS	Frontend UI (Membre 2)	Layouts (AdminLayout, UserLayout, Navbar, Sidebar), composants UI réutilisables, Dashboard Admin, Catalogue produits, Fiche produit	feature/ui- catalogue- dashboard
Manal LAHMER	Modules Transactionnels (Membre 3)	Panier (Zustand cartStore), Commandes (user + admin), Factures (user + admin + auto-création)	feature/panier- commandes- factures

Cette répartition a été conçue pour minimiser les dépendances bloquantes : Aymen a démarré en premier avec le backend et l'authentification, fournissant les API nécessaires aux deux autres

membres. Imad a ensuite implémenté les layouts et composants UI, sur lesquels Manal a pu s'appuyer pour développer les pages transactionnelles.

3. Planification des Tâches

Le projet a débuté le 28 mai 2026 et s'est achevé le 22 juin 2026, soit une durée totale de 26 jours calendaires. Treize tâches ont été identifiées, couvrant l'ensemble du cycle de développement, de l'analyse des besoins initiale à la rédaction du rapport final.

Tableau 2 : Tableau des tâches avec durées et dépendances

#	Tâche	Responsable	Durée	Prédécesseurs
1	Analyse des besoins et conception générale	Aymen	2j	—
2	Setup projet (structure dossiers, env, Git, Vite, Express)	Aymen	1j	1
3	Backend Express + MongoDB + Auth JWT (register, login, middleware)	Aymen	3j	2
4	Gestion Produits + Utilisateurs (backend + frontend admin)	Aymen	2j	3
5	Conception Layouts + Composants UI (AdminLayout, UserLayout, Button, Modal, DataTable)	Imad	2j	3
6	Dashboard Admin + Catalogue Produits (grille, filtres, skeleton)	Imad	2j	5
7	Fiche Produit + Navigation (PrivateRoute, routes imbriquées, App.tsx)	Imad	2j	6
8	Panier — Zustand cartStore (addToCart, removeFromCart, clearCart, totaux)	Manal	3j	4, 5
9	Commandes (création user, liste user, liste admin, approbation/refus)	Manal	4j	8
10	Factures (user + admin + auto-création à l'approbation + populate userId)	Manal	3j	9

11	Intégration des branches (merge, résolution conflits, tests croisés)	Aymen	2j	7, 10
12	Tests et corrections (UTF-8, populate, stock, cartStore, routes UserLayout)	Aymen	2j	11
13	Rédaction du rapport	Tous	2j	12

4. Réseau PERT

Le réseau PERT (Program Evaluation and Review Technique) est un outil de planification qui représente graphiquement les tâches du projet et leurs dépendances sous forme de réseau orienté. Chaque nœud du réseau représente un état du projet (début ou fin d'une tâche), et chaque arc représente une tâche avec sa durée. Le réseau PERT permet d'identifier le chemin critique, c'est-à-dire la séquence de tâches dont la durée cumulée est maximale et qui détermine la durée minimale incompressible du projet.

Dans le réseau PERT de ProduitFlow, chaque nœud est représenté par un cercle numéroté divisé en deux parties : la partie gauche indique la date de début au plus tôt (ES — Early Start) et la partie droite indique la date de début au plus tard (LS — Late Start). La différence entre LS et ES donne la marge totale de chaque tâche.

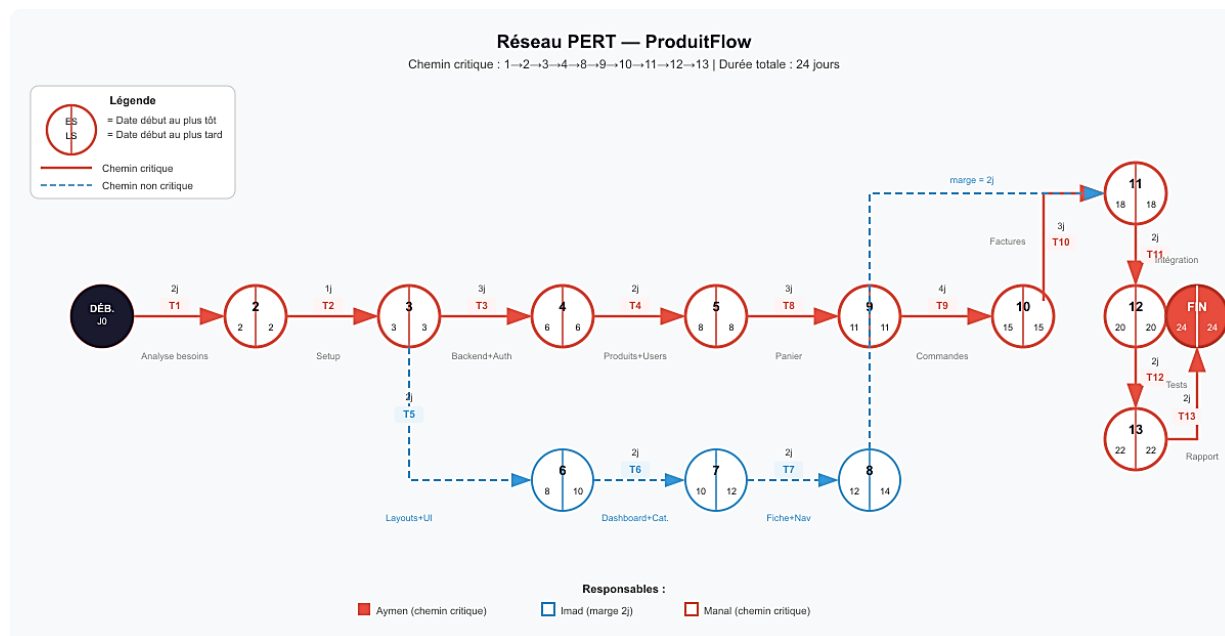


Figure 2 : Réseau PERT du projet

Le chemin critique identifié est : 1 → 2 → 3 → 4 → 8 → 9 → 10 → 11 → 12 → 13, pour une durée totale de 24 jours. Les tâches situées sur ce chemin ont une marge totale nulle, ce qui signifie que tout retard sur l'une d'elles se répercute directement sur la date de fin du projet. Les

tâches 5, 6 et 7 (Imad) disposent d'une marge de 2 jours, ce qui a offert une flexibilité bienvenue lors du développement parallèle.

Tableau 3 : Calcul des marges et chemin critique

Tâche	Durée	ES (plus tôt)	LS (plus tard)	Marge totale	Critique ?
1 — Analyse besoins	2j	J1	J2	0	✓ Oui
2 — Setup projet	1j	J3	J3	0	✓ Oui
3 — Backend + Auth	3j	J4	J6	0	✓ Oui
4 — Produits + Utilisateurs	2j	J7	J8	0	✓ Oui
5 — Layouts + UI	2j	J7	J8	2j	Non
6 — Dashboard + Catalogue	2j	J9	J10	2j	Non
7 — Fiche Produit + Nav	2j	J11	J12	2j	Non
8 — Panier	3j	J9	J11	0	✓ Oui
9 — Commandes	4j	J12	J15	0	✓ Oui
10 — Factures	3j	J16	J18	0	✓ Oui
11 — Intégration	2j	J19	J20	0	✓ Oui
12 — Tests + Corrections	2j	J21	J22	0	✓ Oui
13 — Rapport	2j	J23	J24	0	✓ Oui

5. Diagramme de Gantt

Le diagramme de Gantt complète le réseau PERT en offrant une représentation temporelle du déroulement du projet sur un axe chronologique. Il permet de visualiser simultanément le calendrier de chaque tâche, les chevauchements entre tâches parallèles, et la charge de travail de chaque membre à chaque instant du projet.

Le diagramme de Gantt de ProduitFlow met en évidence trois phases principales : une phase de fondation (tâches 1 à 3) entièrement portée par Aymen, une phase de développement parallèle (tâches 4 à 10) où les trois membres travaillent simultanément sur leurs modules respectifs, et

une phase d'intégration et finalisation (tâches 11 à 13) pilotée par Aymen avec la contribution de tous pour le rapport.

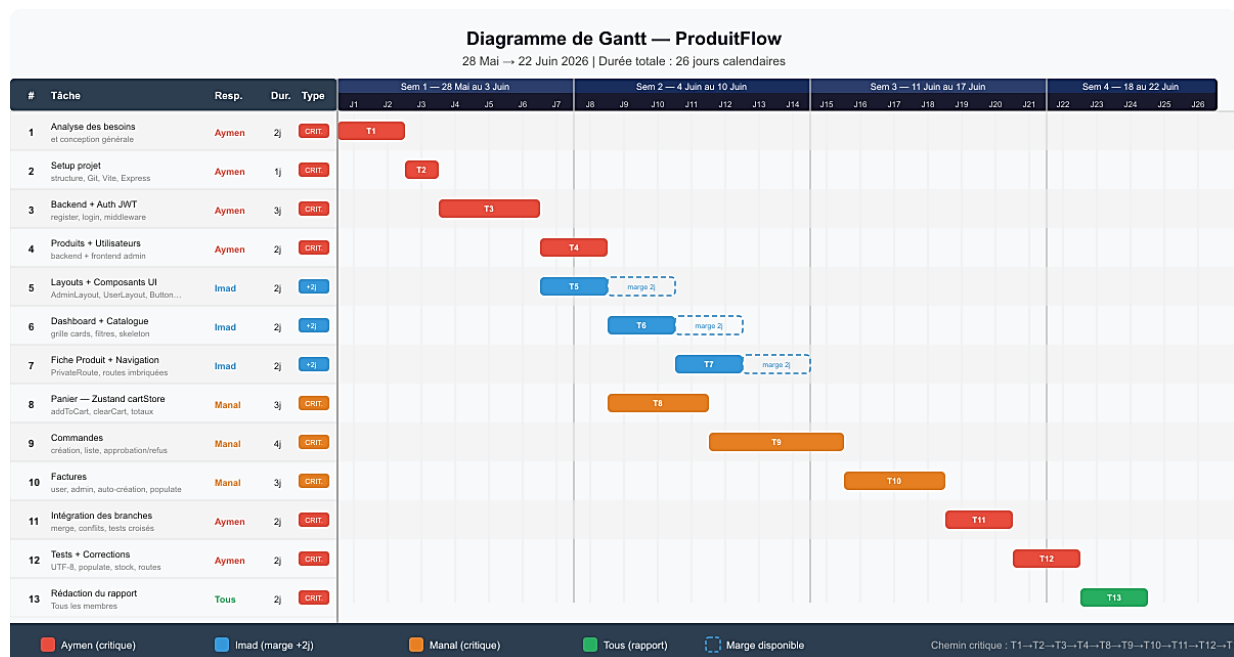


Figure 3 : Diagramme de Gantt du projet

6. Workflow Git

Le dépôt GitHub du projet (<https://github.com/Aymen21-n/ProductFlow>) a été organisé selon un workflow de branches structuré, inspiré du modèle GitFlow adapté à notre contexte académique. Ce workflow garantit que la branche main reste en permanence dans un état stable et déployable.

- Branche main : branche de production, ne recevant que des merges validés via Pull Request.
- feature/auth-produits-utilisateurs (Aymen) : backend complet, authentification JWT, gestion produits et utilisateurs.
- feature/ui-catalogue-dashboard (Imad) : layouts, composants UI, dashboard admin, catalogue et fiche produit.
- feature/panier-commandes-factures (Manal) : modules panier, commandes et factures côté user et admin.

Chaque intégration vers main a suivi le processus suivant : push de la branche feature par le membre concerné, création d'une Pull Request sur GitHub, revue du code par le lead intégrateur (Aymen), identification et résolution des conflits, validation et merge. Cette discipline a permis de détecter et corriger plusieurs problèmes d'intégration importants lors du merge de la branche de Manal : corruption de l'encodage UTF-8, routes user sans UserLayout, incohérence entre

Zustand et localStorage pour le panier, références incorrectes dans les schemas Mongoose pour le populate.

7. Conclusion

La gestion rigoureuse du projet ProduitFlow, appuyée par des outils de planification adaptés (PERT, Gantt) et un workflow Git discipliné, a permis de livrer la totalité des fonctionnalités planifiées dans le délai de 26 jours. La répartition claire des tâches entre les trois membres, la définition explicite des dépendances et la gestion contrôlée des intégrations ont été les facteurs clés du succès de cette collaboration.

Chapitre 3 : Spécification des Besoins

1. Introduction

Ce chapitre constitue la phase de définition formelle des exigences de ProduitFlow. La spécification des besoins est une étape fondamentale du cycle de développement logiciel : elle établit le contrat entre les attentes des utilisateurs finaux et les fonctionnalités que le système doit fournir. Nous distinguons les besoins fonctionnels, qui décrivent les comportements attendus du système, des besoins non fonctionnels, qui définissent les contraintes de qualité, de performance et de sécurité devant être respectées.

2. Identification des Acteurs

ProduitFlow implique deux acteurs principaux, dont les périmètres fonctionnels sont strictement séparés par le système de rôles :

- L'Administrateur : responsable de la gestion opérationnelle de la plateforme. Il dispose d'un accès complet à toutes les entités (produits, commandes, factures, utilisateurs) et est le seul à pouvoir approuver ou refuser les commandes, déclenchant ainsi la décrémentation du stock et la génération automatique de la facture.
- L'Utilisateur : client de la plateforme. Il consulte le catalogue, gère son panier, passe des commandes et consulte ses factures. Il n'a accès qu'à ses propres données et ne peut pas accéder à l'espace d'administration.

3. Besoins Fonctionnels

3.1. Module Authentification

Ce module constitue le point d'entrée de l'application et gère l'ensemble du cycle de vie des sessions utilisateur.

- BF-01 : L'utilisateur peut s'inscrire en fournissant son nom, son email et son mot de passe. Le système valide l'unicité de l'email et hache le mot de passe avec bcryptjs avant stockage.
- BF-02 : L'utilisateur peut se connecter avec son email et son mot de passe. En cas de succès, le système génère un token JWT signé contenant l'identifiant et le rôle de l'utilisateur.

- BF-03 : Le token JWT est stocké dans sessionStorage et automatiquement injecté dans les en-têtes HTTP de toutes les requêtes axios suivantes via la fonction setAuthToken().
- BF-04 : Un flag isInitialized dans AuthContext prévient les redirections intempestives lors du chargement initial de l'application, avant que l'état d'authentification ne soit restauré depuis sessionStorage.
- BF-05 : Le composant PrivateRoute protège toutes les routes sensibles. Il redirige vers /login si l'utilisateur n'est pas authentifié, et vers /unauthorized si le rôle ne correspond pas à l'accès requis.
- BF-06 : L'utilisateur peut se déconnecter. Le système supprime le token de sessionStorage, réinitialise l'état AuthContext et redirige vers la page de connexion.
- 3.2. Module Produits (Admin)
- BF-07 : L'administrateur peut créer un produit en renseignant son nom, sa description, son prix, son stock initial, l'URL de son image et sa catégorie.
- BF-08 : L'administrateur peut modifier les informations d'un produit existant via un modal d'édition pré-rempli.
- BF-09 : L'administrateur peut supprimer un produit avec confirmation.
- BF-10 : La liste des produits est affichée dans un tableau avec toutes les informations et les actions disponibles (modifier, supprimer).

3.3. Module Catalogue (User)

- BF-11 : L'utilisateur consulte le catalogue sous forme de grille de cards affichant pour chaque produit : image, nom, catégorie, prix et stock disponible.
- BF-12 : L'utilisateur peut filtrer le catalogue par nom (recherche en temps réel), par catégorie (dropdown) et par plage de prix (min/max).
- BF-13 : Des skeleton loaders s'affichent pendant le chargement des données pour garantir une expérience fluide.
- BF-14 : Chaque card propose un bouton 'Voir détail' menant à la fiche produit.

3.4. Module Fiche Produit et Panier

- BF-15 : La fiche produit affiche l'image, le nom, la description, la catégorie, le prix et le stock disponible du produit sélectionné.

- BF-16 : L'utilisateur peut sélectionner une quantité via un sélecteur +/- borné entre 1 et le stock disponible, empêchant toute saisie hors plage.
- BF-17 : Le clic sur 'Ajouter au panier' appelle addToCart() du store Zustand. Si le produit est déjà au panier, la quantité est incrémentée. Un message de succès s'affiche pendant 3 secondes.
- BF-18 : La page Panier affiche la liste des articles (image, nom, prix unitaire, quantité modifiable, sous-total, bouton de suppression) et le total général.
- BF-19 : Le panier est géré exclusivement par Zustand (cartStore), sans recours à localStorage.
- BF-20 : Le clic sur 'Passer la commande' envoie les lignes du panier vers l'API, crée la commande avec statut 'en_attente', vide le panier via clearCart() et redirige vers la liste des commandes.

3.5. Module Commandes

- BF-21 : L'utilisateur consulte la liste de ses commandes triées par date décroissante, avec le statut affiché en badge coloré (en attente : orange, approuvée : vert, refusée : rouge).
- BF-22 : Un modal de détail affiche les lignes de la commande, les quantités et le montant total.
- BF-23 : L'administrateur consulte toutes les commandes de tous les utilisateurs, avec le nom de l'utilisateur résolu via .populate('userId', 'nom') côté backend.
- BF-24 : L'administrateur peut filtrer les commandes par nom d'utilisateur.
- BF-25 : L'administrateur peut approuver ou refuser une commande. Les boutons sont désactivés pour les commandes déjà traitées.
- BF-26 : À l'approbation, le backend décrémente le stock de chaque produit commandé via \$inc { stock: -quantite } et crée automatiquement une Facture.

3.6. Module Factures

- BF-27 : L'utilisateur consulte la liste de ses factures générées automatiquement après approbation de ses commandes.
- BF-28 : L'administrateur consulte toutes les factures avec le nom de l'utilisateur (populate) et peut les filtrer par nom.
- BF-29 : Un modal de détail affiche les lignes de la facture et le montant total.

3.7. Module Utilisateurs (Admin)

- BF-30 : L'administrateur consulte la liste complète des utilisateurs avec leur nom, email, rôle et statut (actif/bloqué).
- BF-31 : L'administrateur peut bloquer ou débloquent un utilisateur.
- BF-32 : L'administrateur peut supprimer un utilisateur.

3.8. Tableau de Bord Admin

- BF-33 : Le dashboard affiche 4 indicateurs clés : nombre total de commandes, revenus totaux (filtrés sur les commandes au statut 'approuvée' uniquement), nombre de produits en stock faible (stock < 5), nombre total d'utilisateurs.
- BF-34 : Des skeleton loaders s'affichent pendant le chargement des statistiques.
- BF-35 : Des raccourcis de navigation rapide permettent d'accéder directement aux sections principales.

Tableau 4 : Récapitulatif des besoins fonctionnels par module

ID	Besoin Fonctionnel	Acteur	Priorité
BF-01 à 06	Module Authentification (inscription, connexion, déconnexion, JWT, PrivateRoute)	Admin + User	Haute
BF-07 à 10	Gestion des Produits CRUD	Admin	Haute
BF-11 à 14	Catalogue Produits avec filtres et skeleton	User	Haute
BF-15 à 20	Fiche Produit, sélecteur quantité, Panier Zustand	User	Haute
BF-21 à 26	Commandes : création, suivi user, gestion admin, approbation, stock	Admin + User	Haute
BF-27 à 29	Factures : auto-génération, liste user, liste admin, détail	Admin + User	Haute
BF-30 à 32	Gestion des Utilisateurs	Admin	Moyenne
BF-33 à 35	Tableau de Bord Admin avec statistiques	Admin	Haute

4. Besoins Non Fonctionnels

4.1. Performance

- BNF-01 : Les pages principales doivent se charger en moins de 2 secondes en conditions normales d'utilisation.
- BNF-02 : Des skeleton loaders sont affichés pendant tous les appels API pour maintenir une expérience utilisateur fluide, sans affichage d'écran vide.
- BNF-03 : L'instance axios partagée avec timeout de 5 000 ms prévient les blocages indéfinis en cas de non-réponse du backend.

4.2. Sécurité

- BNF-04 : Les mots de passe sont hachés avec bcryptjs (salt rounds 10) avant stockage en base de données.
- BNF-05 : L'authentification repose sur des tokens JWT signés avec une clé secrète stockée dans les variables d'environnement (.env non versionné).
- BNF-06 : Tous les endpoints sensibles du backend sont protégés par le middleware auth.ts qui vérifie la validité et l'expiration du token JWT.
- BNF-07 : Les routes frontend sont protégées par PrivateRoute selon le rôle de l'utilisateur authentifié.

4.3. Maintenabilité

- BNF-08 : Tous les types TypeScript partagés entre frontend et backend sont centralisés dans src/types/index.ts, constituant la source unique de vérité.
- BNF-09 : Toutes les requêtes API transitent par une instance axios partagée (src/api/axios.ts), facilitant la configuration centralisée des intercepteurs et des en-têtes.
- BNF-10 : L'architecture frontend respecte une séparation claire entre pages, composants, services et store.

4.4. Ergonomie et Accessibilité

- BNF-11 : L'interface s'adapte aux différentes résolutions d'écran (responsive design).
- BNF-12 : Les messages d'erreur et de succès sont clairs, explicites et temporisés (disparaissent après 3 secondes).
- BNF-13 : Les statuts des commandes sont visualisés par des badges colorés immédiatement compréhensibles.

5. Diagrammes de Cas d'Utilisation

Les diagrammes de cas d'utilisation UML décrivent les interactions entre les acteurs (Administrateur, Utilisateur) et le système ProduitFlow. Ils constituent la vue fonctionnelle externe du système et servent de base à la conception des interfaces graphiques.

Tableau 5 : Description des cas d'utilisation principaux

Cas d'utilisation	Acteur	Description	Précondition
S'inscrire	Utilisateur	Créer un compte avec nom, email, mot de passe	Aucune
Se connecter	Admin + User	Authentification par email/mot de passe + JWT	Compte existant
Consulter le catalogue	Utilisateur	Parcourir et filtrer les produits disponibles	Authentifié (rôle user)
Ajouter au panier	Utilisateur	Ajouter un produit avec quantité choisie	Authentifié (rôle user)
Passer une commande	Utilisateur	Valider le contenu du panier	Panier non vide
Consulter commandes	Admin + User	Voir liste des commandes + détails	Authentifié
Approuver/Refuser commande	Administrateur	Changer le statut d'une commande	Authentifié (rôle admin)
Gérer les produits	Administrateur	CRUD complet sur le catalogue	Authentifié (rôle admin)
Consulter le dashboard	Administrateur	Voir les statistiques clés	Authentifié (rôle admin)
Consulter factures	Admin + User	Voir les factures générées automatiquement	Authentifié

Diagramme de Cas d'Utilisation — Espace Administrateur

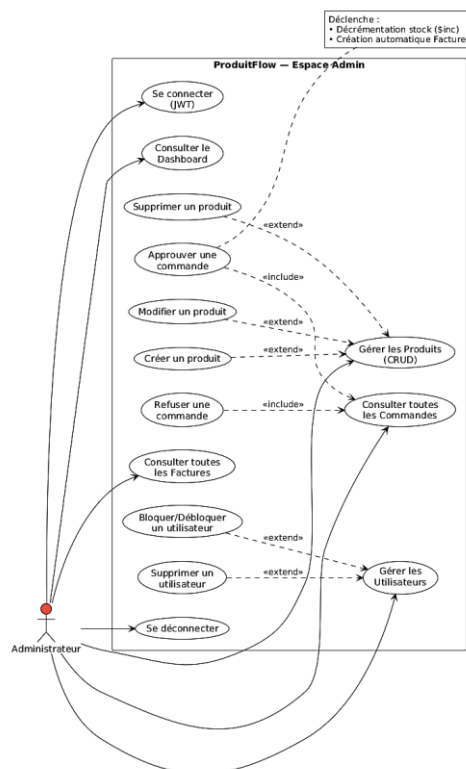


Figure 4 : Diagramme de cas d'utilisation — Espace Administrateur

Diagramme de Cas d'Utilisation — Espace Utilisateur

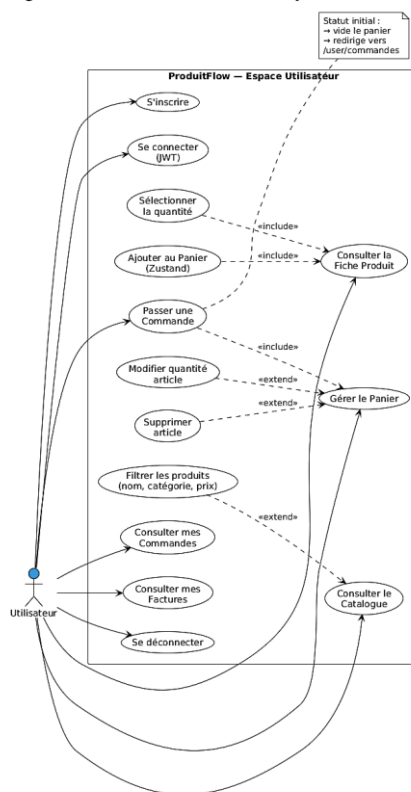


Figure 5 : Diagramme de cas d'utilisation — Espace Utilisateur

6. Conclusion

Cette spécification formelle des besoins constitue le référentiel fonctionnel de ProduitFlow. Les 35 besoins fonctionnels et 13 besoins non fonctionnels identifiés couvrent l'ensemble des fonctionnalités attendues et des contraintes de qualité à respecter. Ce référentiel a guidé toutes les décisions de conception et de réalisation présentées dans les chapitres suivants, garantissant la cohérence entre les exigences exprimées et la solution technique livrée.

Chapitre 4 : Conception du Système

1. Introduction

Ce chapitre présente la traduction des besoins fonctionnels identifiés en une architecture technique concrète et cohérente. La conception de ProduitFlow a suivi une approche en deux dimensions complémentaires : la modélisation dynamique, qui décrit les comportements et interactions du système dans le temps, et la modélisation statique, qui définit la structure des données et l'organisation architecturale du code. L'ensemble des diagrammes a été produit avec l'outil PlantUML, offrant une modélisation reproductible et maintenable.

2. Modélisation Dynamique

2.1. Diagrammes de Séquences

Les diagrammes de séquences UML modélisent les échanges chronologiques entre les acteurs et les composants du système pour les scénarios fonctionnels les plus critiques. Chaque diagramme met en évidence les messages synchrones et asynchrones échangés, les traitements effectués à chaque étape et les données transitant entre les couches de l'application.

Diagramme 1 — Authentification (Login) :

Ce scénario décrit le flux complet depuis la saisie des identifiants jusqu'à l'accès au dashboard approprié. L'acteur principal est le composant Login.tsx qui interagit avec AuthContext, le service authService, l'instance axios, le backend Express et la base MongoDB.

- L'utilisateur saisit son email et mot de passe dans le formulaire Login.tsx (React Hook Form).
- AuthContext reçoit l'action et dispatche LOGIN_START, passant loading à true.
- authService.login() est appelé, envoyant une requête POST /api/auth/login via l'instance axios.
- Le contrôleur authController.ts vérifie l'existence de l'utilisateur en base, compare le mot de passe fourni avec le hash bcryptjs stocké, et génère un token JWT signé avec la clé secrète.
- En cas de succès, LOGIN_SUCCESS est dispatché avec { user, token }, le token est stocké dans sessionStorage, setAuthToken() configure l'en-tête Authorization des requêtes axios suivantes.

- PrivateRoute détecte le changement d'état et redirige vers /admin/dashboard ou /user/catalogue selon le rôle.

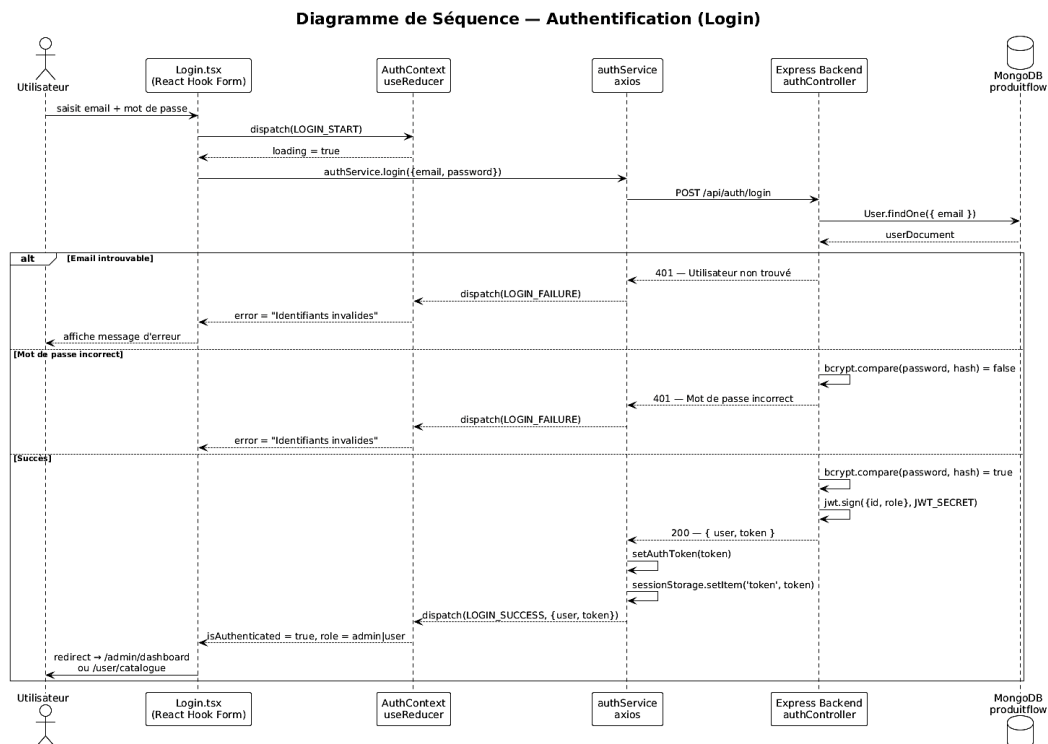


Figure 6 : Diagramme de séquence — Authentification (Login)

Diagramme 2 — Passer une Commande (Utilisateur) :

Ce scénario décrit la création d'une commande depuis la page Panier jusqu'à la confirmation.

- L'utilisateur consulte son panier (données issues de Zustand cartStore) et clique sur 'Passer la commande'.
- Panier.tsx appelle commandeService.createCommande() en transmettant les lignes du panier (LignePanier[]), l'identifiant de l'utilisateur et le montant total calculé.
- Le backend reçoit la requête POST /api/commandes, vérifie le token JWT via le middleware auth.ts, puis crée l'entrée Commande en base avec statut 'en_attente'.
- La réponse de succès est reçue, clearCart() vide le store Zustand et l'utilisateur est redirigé vers /user/commandes.

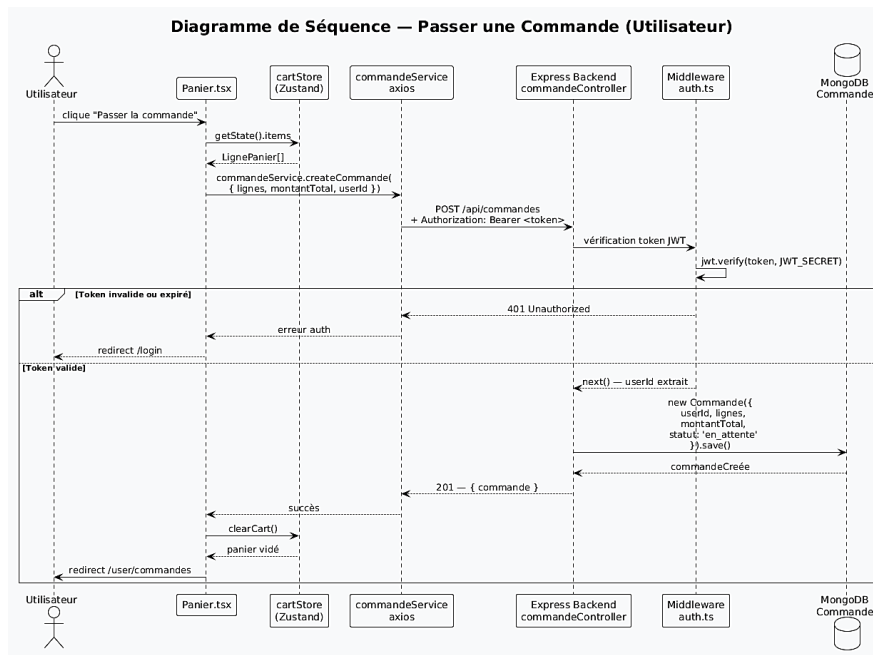


Figure 7 : Diagramme de séquence — Passer une commande

Diagramme 3 — Approuver une Commande (Administrateur) :

Ce scénario est le plus complexe car il implique trois opérations atomiques déclenchées en cascade.

- L'administrateur clique sur 'Approuver' dans la page Commandes Admin.
- `commandeService.updateStatut()` envoie PUT `/api/commandes/:id/statut` avec `{ statut: 'approuvee' }`.
- Le contrôleur `commandeController.ts` met à jour le statut de la commande, puis itère sur chaque ligne de la commande pour décrémenter le stock via `Produit.findByIdAndUpdate(ligne.produitId, { $inc: { stock: -ligne.quantite } })`.
- Une Facture est automatiquement créée en base, reprenant les lignes de la commande, le `userId`, le montant total et la date.
- La réponse de succès est renvoyée, l'interface admin se met à jour et les boutons Approuver/Refuser sont désactivés.

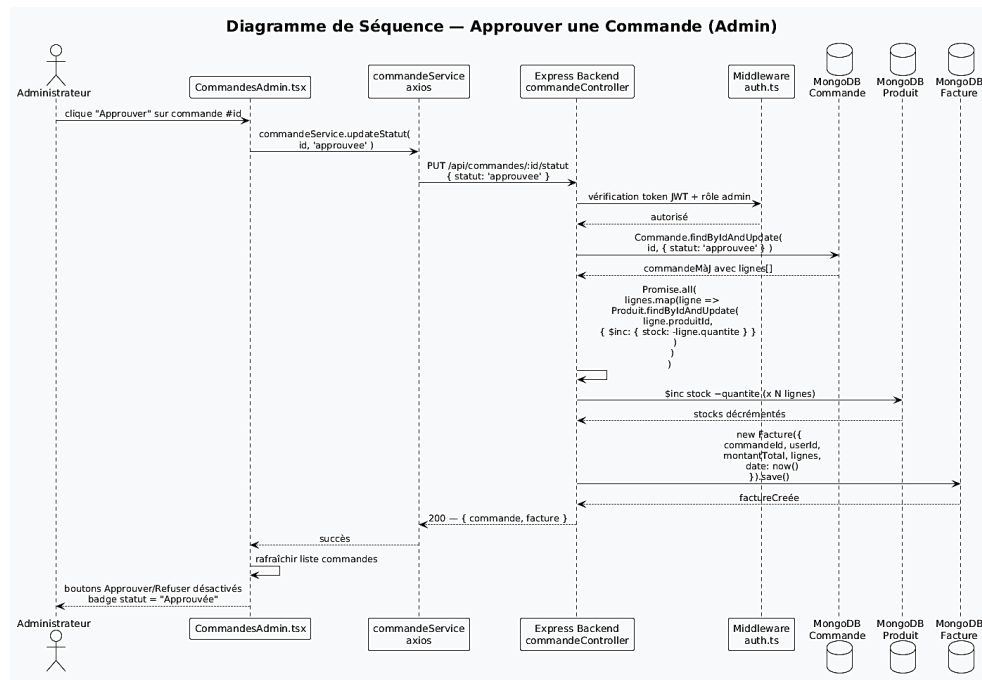


Figure 8 : Diagramme de séquence — Approuver une commande (Admin)

2.2. Diagramme d'Activité

Le diagramme d'activité modélise le flux de traitement global d'une commande dans ProduitFlow, depuis l'ajout d'un produit au panier jusqu'à la consultation de la facture générée. Ce diagramme illustre les points de décision, les actions parallèles et les responsabilités de chaque acteur à chaque étape du processus.

- Phase 1 — Sélection produit : l'utilisateur parcourt le catalogue, consulte la fiche produit, sélectionne la quantité souhaitée (bornée au stock) et ajoute au panier.
- Phase 2 — Validation panier : l'utilisateur vérifie le contenu de son panier, ajuste éventuellement les quantités et confirme la commande.
- Phase 3 — Traitement backend : le backend crée la commande en base avec statut 'en_attente'. L'administrateur est notifié (via le dashboard) de la nouvelle commande en attente.
- Phase 4 — Décision admin : l'administrateur approuve ou refuse la commande. Si refus, la commande passe au statut 'refusee' et l'utilisateur en est informé. Si approbation, trois actions atomiques s'enchaînent : mise à jour du statut, décrémentation du stock, création de la facture.
- Phase 5 — Consultation : l'utilisateur peut consulter sa commande approuvée et la facture générée automatiquement.

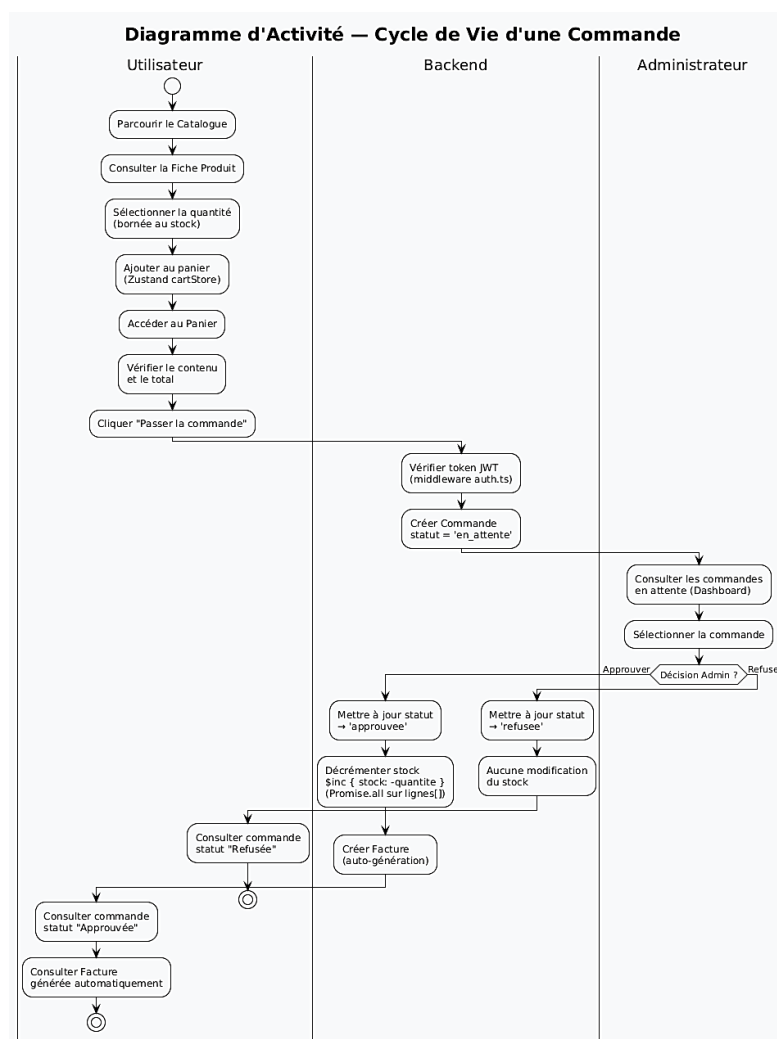


Figure 9 : Diagramme d'activité — Cycle de vie complet d'une commande

3. Modélisation Statique

3.1. Diagramme de Classes

Le diagramme de classes définit la structure statique de l'application en identifiant les entités métier, leurs attributs, leurs méthodes et leurs relations. Dans ProduitFlow, les classes correspondent directement aux modèles Mongoose du backend et aux interfaces TypeScript partagées du frontend.

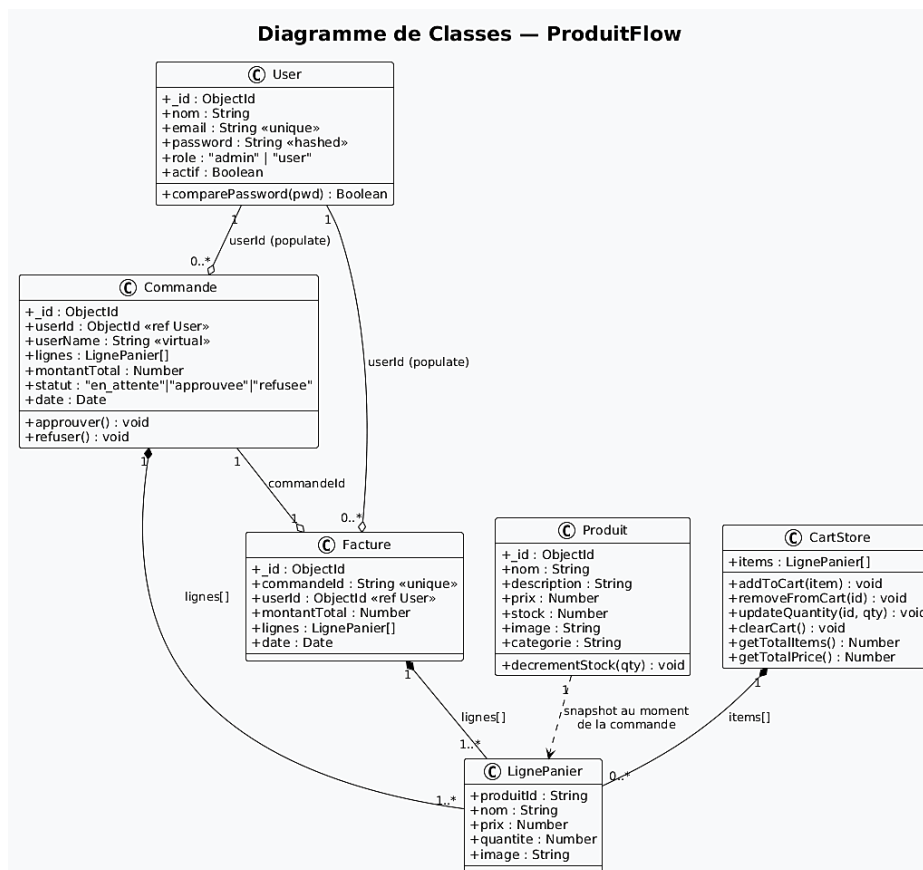


Figure 10 : Diagramme de classes — ProduitFlow

Les cinq classes principales et leurs relations sont les suivantes :

- **User** : { `_id`: ObjectId, `nom`: String, `email`: String (unique), `password`: String (hashé), `role`: 'admin'|'user', `actif`: Boolean }. Classe centrale référencée par **Commande** et **Facture** via `populate`.
- **Produit** : { `_id`: ObjectId, `nom`: String, `description`: String, `prix`: Number, `stock`: Number, `image`: String, `categorie`: String }. Le stock est décrémenté automatiquement à l'approbation d'une commande.
- **LignePanier** (sous-document) : { `produitId`: String, `nom`: String, `prix`: Number, `quantite`: Number, `image`: String }. Embarqué dans **Commande** et **Facture**, il constitue l'instantané des informations produit au moment de la transaction.
- **Commande** : { `_id`: ObjectId, `userId`: ObjectId (ref User), `lignes`: LignePanier[], `montantTotal`: Number, `statut`: 'en_attente'|'approuvee'|'refusee', `date`: Date }. La relation avec **User** est résolue par `.populate('userId', 'nom')` dans les routes admin.

- Facture : { _id: ObjectId, commandeId: String (unique), userId: ObjectId (ref User), montantTotal: Number, lignes: LignePanier[], date: Date }. Créée automatiquement par le backend à l'approbation d'une commande.

3.2. Types TypeScript Partagés

Conformément à la décision architecturale prise en début de projet, tous les types TypeScript partagés entre les composants frontend sont centralisés dans le fichier `src/types/index.ts`. Ce fichier constitue la source unique de vérité pour les interfaces de l'application et garantit la cohérence du typage à travers l'ensemble du codebase.

Tableau 6 : Types partagés — `src/types/index.ts`

Interface	Champs principaux	Usage dans l'application
User	id, nom, email, password, role, actif	AuthContext, authReducer, gestion utilisateurs admin
Produit	id, nom, description, prix, stock, image, categorie	Catalogue, fiche produit, CRUD admin, cartStore
LignePanier	produitId, nom, prix, quantite, image	cartStore Zustand, création commande, affichage panier
Commande	id, userId, userName?, lignes, montantTotal, statut, date	Pages commandes user + admin, dashboard
Facture	id, commandeId, userId (string { _id, nom }), montantTotal, lignes, date	Pages factures user + admin

3.3. Architecture de l'Application

L'architecture de ProduitFlow suit un modèle client-serveur à trois couches avec une séparation nette des responsabilités entre présentation, logique métier et persistance des données.

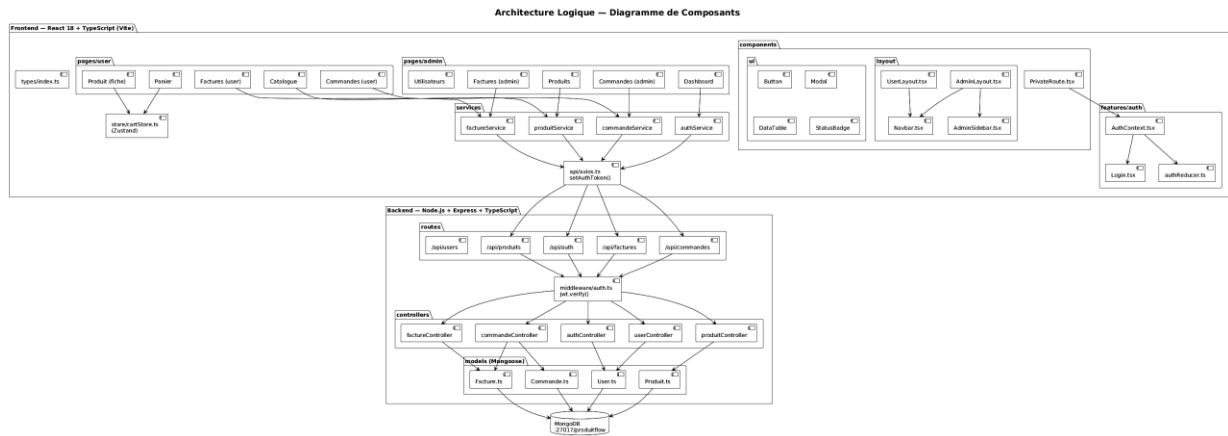


Figure 11 :: Architecture logique — Diagramme de composants

Couche Présentation (Frontend) : React 18 + TypeScript, structuré en modules fonctionnels. Les pages admin/ et user/ sont organisées en dossiers séparés, chacun avec son propre layout wrapper. Les composants ui/ sont réutilisables entre les deux espaces. Les services/ centralisent les appels API par domaine fonctionnel. Le store Zustand (cartStore) gère l'état global du panier.

Couche API (Backend) : Express.js + TypeScript avec architecture MVC. Les routes/ définissent les endpoints RESTful protégés par le middleware auth.ts. Les controllers/ implémentent la logique métier. Les models/ définissent les schemas Mongoose avec leurs validations et transformations.

Couche Données : MongoDB local sur le port 27017, base nommée 'produitflow'. Les relations entre entités (Commande $\rightarrow$ User, Facture $\rightarrow$ User) sont gérées par des références ObjectId avec résolution par populate lors des requêtes admin.

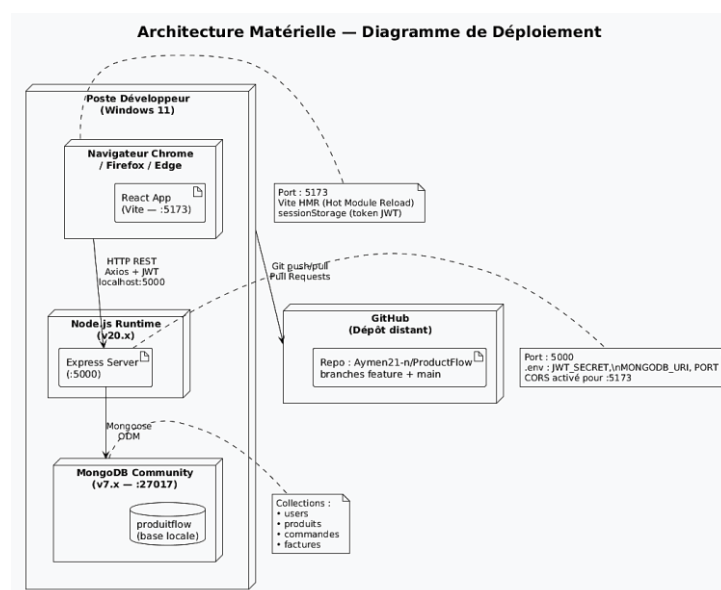


Figure 12 : Architecture matérielle — Diagramme de déploiement

3.4. Structure du Frontend

La structure du projet frontend respecte une organisation claire favorisant la maintenabilité et la séparation des responsabilités :

Tableau 7 : Structure du frontend — organisation des dossiers src/

Dossier / Fichier	Contenu / Responsabilité
src/api/axios.ts	Instance axios partagée avec baseURL, timeout 5000ms, et setAuthToken()
src/features/auth/	AuthContext.tsx (Provider + useAuth hook), authReducer.ts (actions + state), Login.tsx
src/components/layout/	AdminLayout.tsx, UserLayout.tsx, Navbar.tsx, AdminSidebar.tsx
src/components/ui/	Button, StatusBadge, Modal, DataTable — composants réutilisables
src/components/PrivateRoute.tsx	Protection des routes selon authentification et rôle
src/pages/admin/	Dashboard, Produits, Commandes, Factures, Utilisateurs — espace admin
src/pages/user/	Catalogue, Produit, Panier, Commandes, Factures — espace utilisateur
src/services/	authService, produitService, commandeService, factureService — appels API
src/store/cartStore.ts	Store Zustand pour le panier (items, addToCart, removeFromCart, updateQuantity, clearCart)
src/types/index.ts	Source unique de vérité pour les interfaces TypeScript partagées

3.5. Routes Backend Complètes

Tableau 8 : Routes Backend RESTful — récapitulatif complet

Méthode	Endpoint	Contrôleur	Description	Accès
POST	/api/auth/register	authController	Inscription + hachage mot de passe	Public
POST	/api/auth/login	authController	Login + génération token JWT	Public

GET	/api/produits	produitController	Liste tous les produits	Authentifié
GET	/api/produits/:id	produitController	Détail d'un produit	Authentifié
POST	/api/produits	produitController	Créer un nouveau produit	Admin
PUT	/api/produits/:id	produitController	Modifier un produit	Admin
DELETE	/api/produits/:id	produitController	Supprimer un produit	Admin
GET	/api/commandes	commandeController	Toutes commandes + populate('userId','nom')	Admin
GET	/api/commandes/user/:userId	commandeController	Commandes d'un utilisateur spécifique	User
POST	/api/commandes	commandeController	Créer une commande depuis le panier	User
PUT	/api/commandes/:id/statut	commandeController	Approuver/Refuser + \$inc stock + créer Facture	Admin
GET	/api/factures	factureController	Toutes factures + populate('userId','nom')	Admin
GET	/api/factures/user/:userId	factureController	Factures d'un utilisateur	User
GET	/api/factures/:id	factureController	Détail d'une facture	Authentifié
GET	/api/users	UserController	Liste tous les utilisateurs	Admin
PUT	/api/users/:id/bloquer	UserController	Basculer statut actif/bloqué	Admin
DELETE	/api/users/:id	UserController	Supprimer un utilisateur	Admin

4. Conclusion

La phase de conception a permis de traduire l'ensemble des 35 besoins fonctionnels en une architecture technique précise et cohérente. Les modèles UML produits — diagrammes de séquences, diagramme d'activité, diagramme de classes — constituent une documentation technique complète qui a guidé l'implémentation et facilité la communication au sein de l'équipe. L'architecture en trois couches avec séparation nette des responsabilités offre une base solide pour les évolutions futures de la plateforme.

Chapitre 5 : Réalisation du Système

1. Introduction

Ce chapitre présente la mise en œuvre concrète de ProduitFlow, en décrivant l'environnement de développement mis en place, les interfaces graphiques des fonctionnalités principales et les points techniques clés qui ont caractérisé le développement. La réalisation a traduit fidèlement les choix de conception exposés dans le chapitre précédent, tout en nécessitant plusieurs ajustements techniques lors de la phase d'intégration.

2. Environnement de Développement

2.1. Environnement Matériel

- Postes de développement : trois machines Windows 11 avec processeurs Intel Core i5/i7 et 8 à 16 GB de RAM.
- Outils de test : navigateurs Chrome (DevTools), Firefox et Edge pour les tests de compatibilité cross-browser.
- Gestion de version : dépôt GitHub (<https://github.com/Aymen21-n/ProductFlow>) avec workflow de branches feature.
- Base de données locale : MongoDB Community Server 7.x installé localement sur le port 27017, base 'produitflow'.

2.2. Stack Technique Complète

Tableau 9 : Stack technique — environnement de développement

Technologie / Outil	Rôle dans le Projet	Version
React 18	Framework UI — composants fonctionnels, hooks, rendering	18.x
TypeScript	Typage statique JavaScript — frontend et backend	5.x
Vite	Bundler ultra-rapide et serveur de développement	5.x
React Router DOM v7	Routing SPA — routes imbriquées, Outlet, Navigate	7.x
Zustand	Store global léger pour le panier (cartStore)	4.x
Axios	Client HTTP avec instance partagée et intercepteurs	1.x
React Hook Form	Gestion des formulaires avec validation schema	7.x

React Icons	Bibliothèque d'icônes SVG inline	5.x
Node.js	Runtime JavaScript côté serveur	20.x
Express.js	Framework serveur web RESTful	4.x
Mongoose	ODM MongoDB — modèles, schémas, populate, \$inc	8.x
MongoDB	Base de données NoSQL orientée documents	7.x
bcryptjs	Hachage sécurisé des mots de passe (salt rounds 10)	2.x
JSON Web Token	Authentification stateless par token signé	9.x
VS Code	IDE principal — extensions TypeScript, ESLint, Prettier	Latest
GitHub	Contrôle de version, Pull Requests, historique commits	—
MongoDB Compass	Interface graphique pour l'exploration de la base	1.x

2.2.1. Frontend :

Le frontend de ProduitFlow repose sur React 18 associé à TypeScript pour un typage statique rigoureux, et Vite comme bundler ultra-rapide. La navigation est gérée par React Router DOM v7 avec routes imbriquées et Outlet. La gestion de l'état global du panier utilise Zustand, tandis que les formulaires sont contrôlés via React Hook Form. Les appels HTTP transitent par une instance Axios partagée, et les icônes sont fournies par React Icons.

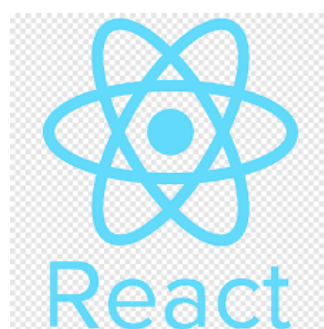


Figure 13 : Logo React



Figure 14 : Logo Zustand



Figure 15 : Logo Vite



Figure 16 : Logo TypeScript



Figure 17 : Logo AXIOS

2.2.2. Backend :

Le backend repose sur Node.js comme runtime JavaScript côté serveur, associé au framework Express.js pour la définition des routes RESTful. La persistance des données est assurée par MongoDB via l'ODM Mongoose, qui gère les schémas, validations et relations entre collections (populate). La sécurisation des mots de passe utilise bcryptjs et l'authentification stateless est implémentée avec JSON Web Token (JWT).



Figure 18 : Logo node.js



Figure 19 : Logo mongoose



Figure 20 : Logo mongoDB



Figure 21 : Logo JWT



Figure 22 : Logo express

2.2.3. Outils de développement :

Le développement a été réalisé sur VS Code avec les extensions TypeScript, ESLint et Prettier. La gestion de version et la collaboration en équipe ont été assurées via GitHub, avec un workflow de branches feature et Pull Requests. L'exploration et la visualisation de la base de données ont été facilitées par MongoDB Compass. Le projet est hébergé publiquement sur le dépôt github.com/Aymen21-n/ProductFlow.



Visual Studio Code

Figure 23 : Logo Visual Studio Code



Figure 24 : Logo MongoDB Compass



Figure 25 : Logo GitHub

3. Variables d'Environnement Backend

Le fichier .env du backend (non versionné sur GitHub) contient les variables d'environnement sensibles suivantes :

- PORT=5000 : port d'écoute du serveur Express.
- MONGODB_URI=mongodb://localhost:27017/produitflow : URI de connexion à la base MongoDB locale.
- JWT_SECRET=produitflow_secret_2024 : clé secrète pour la signature des tokens JWT.

Les comptes de test créés lors du développement sont : admin@produitflow.com / admin123 (rôle admin) et aymen@gmail.com / aymen123 (rôle user).

4. Interfaces Graphiques Principales

4.1. Authentification

La page de connexion et d'inscription est accessible depuis la route /login. Elle gère les deux cas d'usage sur une même interface. Après authentification réussie, l'utilisateur est automatiquement

redirigé vers son espace selon le rôle retourné par le backend : l'administrateur est envoyé vers /admin/dashboard et l'utilisateur vers /user/catalogue. En cas d'erreur (mauvais identifiants, email inexistant), un message d'erreur explicite s'affiche sous le formulaire.

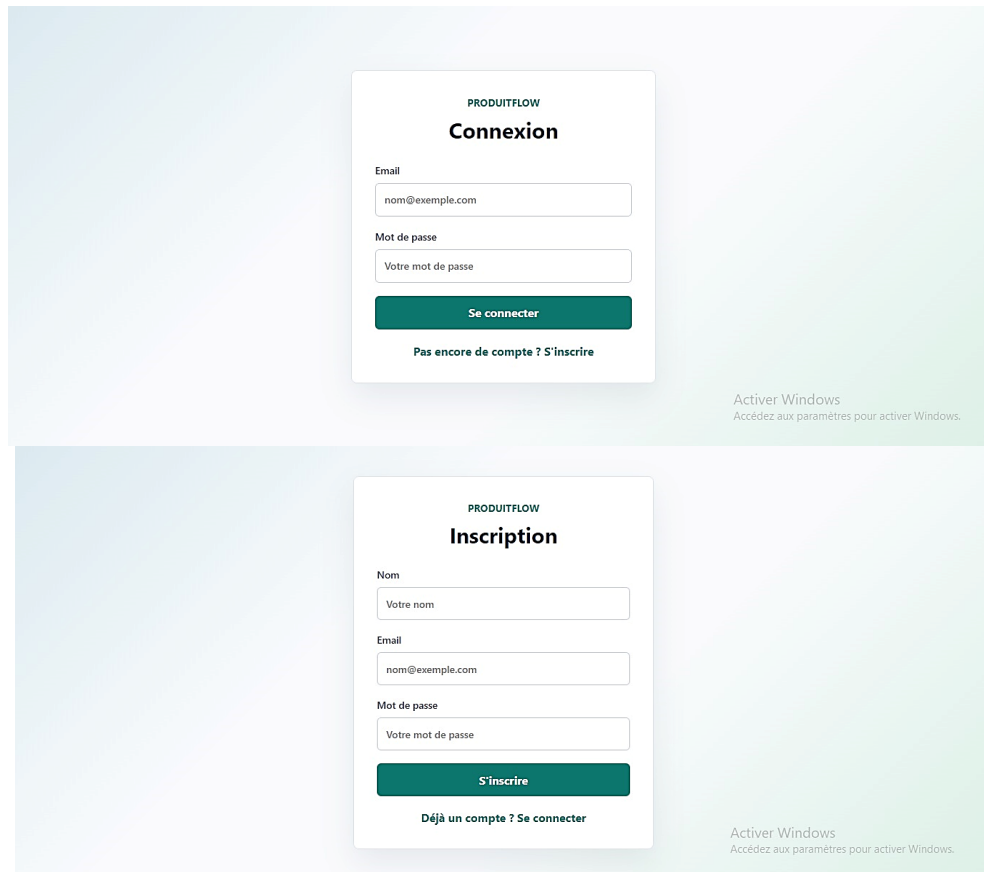


Figure 26 : Page de connexion / inscription

4.2. Espace Administrateur — AdminLayout

L'espace administrateur est accessible via le composant AdminLayout.tsx, qui encapsule toutes les routes admin dans une structure à deux colonnes : une sidebar de navigation à gauche listant les cinq sections de l'espace admin, et une zone de contenu principale à droite rendue via `<Outlet />` selon la route active.

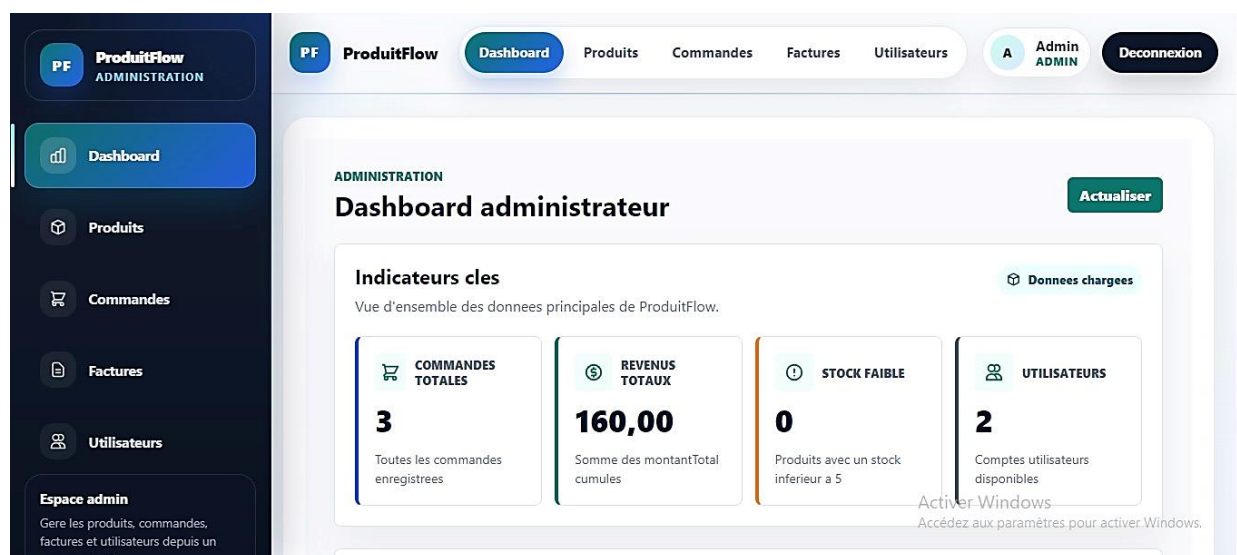


Figure 27 : Dashboard Administrateur — indicateurs clés

Le tableau de bord administrateur affiche quatre indicateurs clés calculés côté frontend à partir des données chargées via les services API : le nombre total de commandes, les revenus totaux filtrés sur les commandes au statut 'approuvée' uniquement (`commandes.filter(c => c.statut === 'approuvée').reduce(...)`), le nombre de produits dont le stock est inférieur à 5 unités, et le nombre total d'utilisateurs. Des raccourcis de navigation rapide permettent d'accéder directement à chaque section.

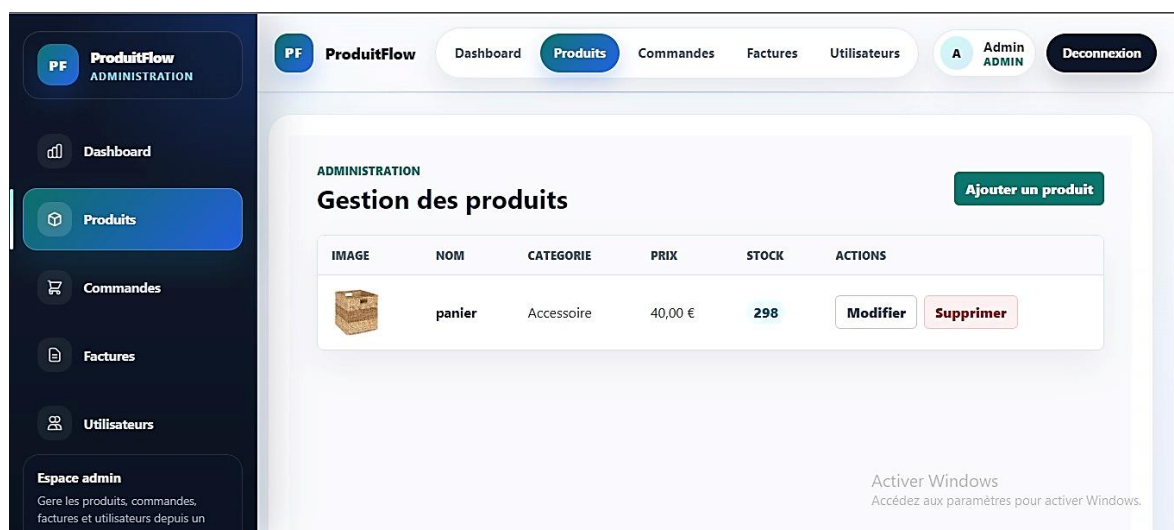


Figure 28 : Page Produits Admin — CRUD complet avec modal

Figure 15

La page de gestion des produits affiche l'ensemble du catalogue dans un tableau récapitulatif. Un bouton 'Nouveau Produit' ouvre un modal de création avec les champs : nom, description, prix, stock, URL de l'image et catégorie. Chaque ligne du tableau propose des boutons de modification

(modal pré-rempli) et de suppression avec confirmation. Les données sont rechargées automatiquement après chaque opération.

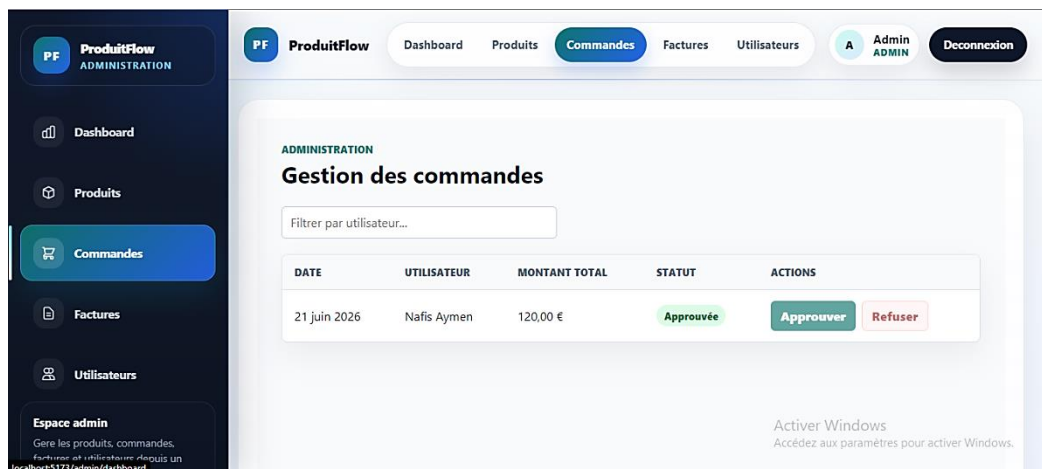


Figure 29 : Page Commandes Administrateur — approbation et filtrage

La page des commandes admin affiche l'ensemble des commandes de tous les utilisateurs, triées par date décroissante. La colonne 'Utilisateur' affiche le nom résolu via `.populate('userId', 'nom')` côté backend, géré par le helper `getUserNom()` côté frontend qui supporte les deux cas de retour (string brut ou objet populé). Un champ de recherche permet de filtrer par nom d'utilisateur. Les boutons Approuver et Refuser sont désactivés pour les commandes déjà traitées.

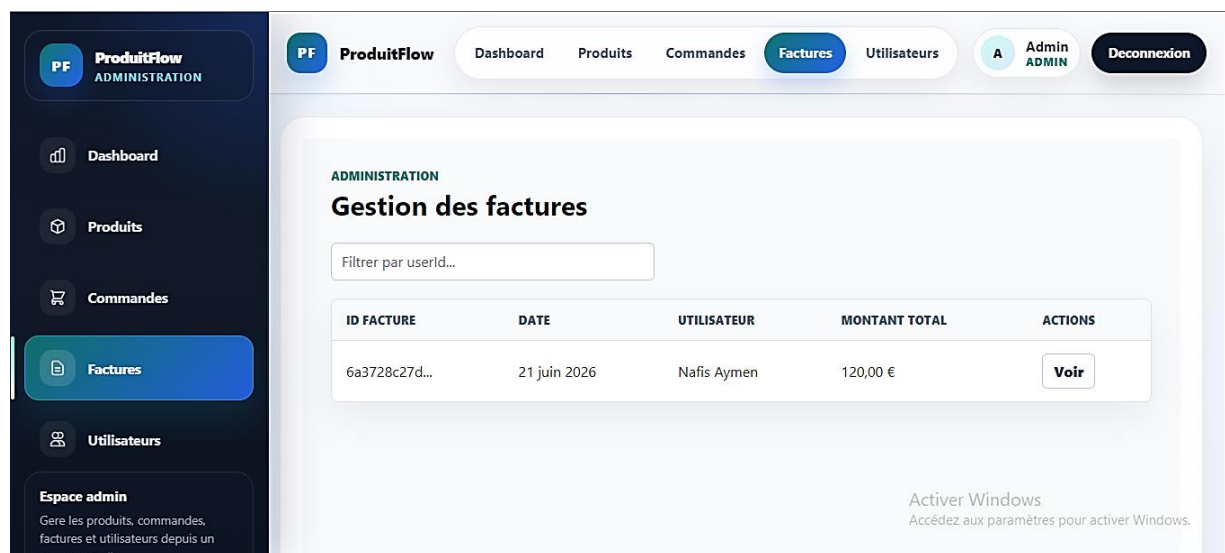


Figure 30 : Page Factures Administrateur

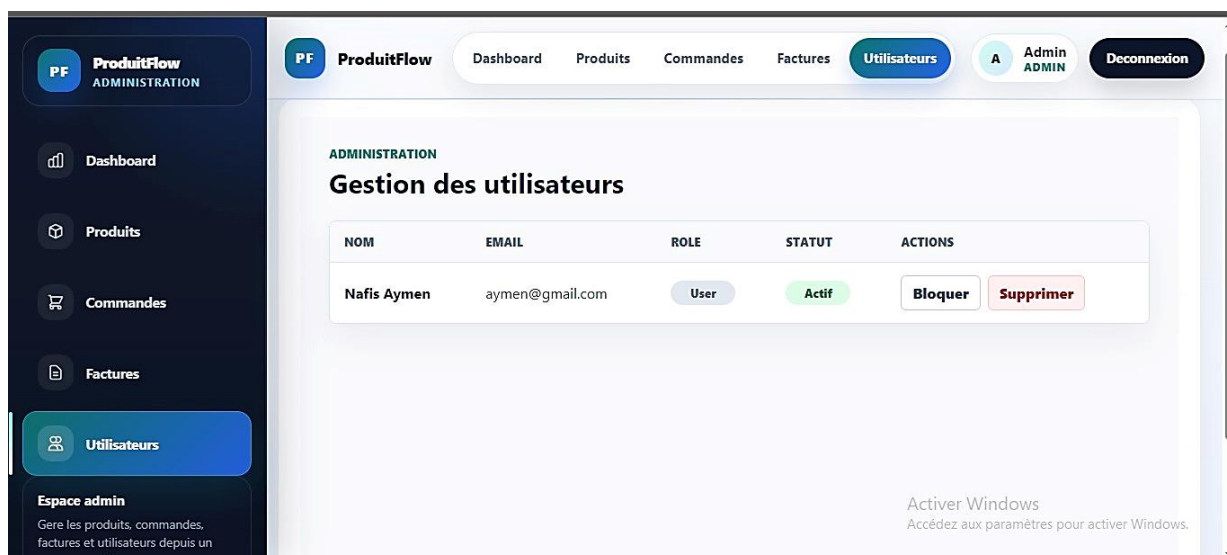


Figure 31 : : Page Utilisateurs Administrateur — blocage et suppression

4.3. Espace Utilisateur — UserLayout

L'espace utilisateur est accessible via UserLayout.tsx, qui propose une navbar horizontale en haut de page avec les liens de navigation (Catalogue, Panier, Commandes, Factures), le nom de l'utilisateur connecté et le bouton de déconnexion. Le contenu des pages est rendu via `<Outlet />` selon la route active.

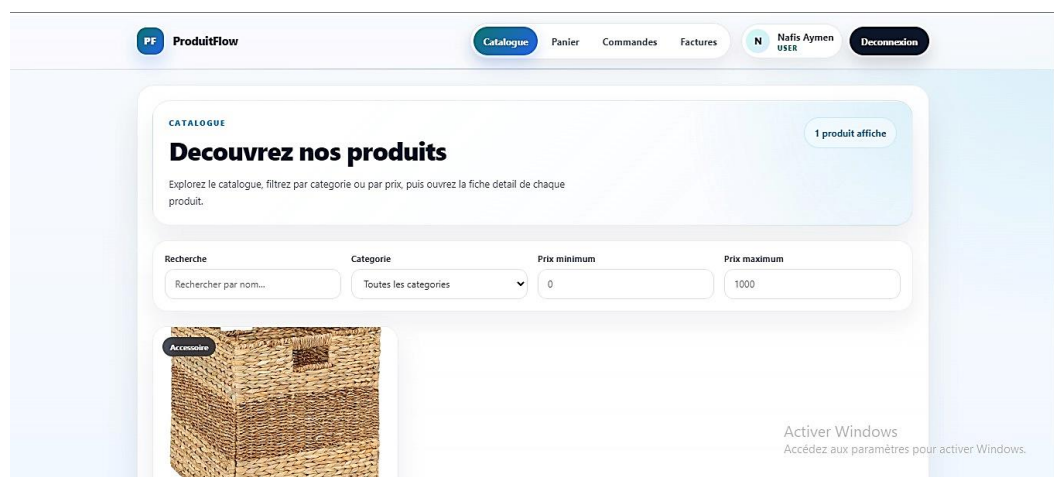


Figure 32 : Catalogue Produits — grille avec filtres avancés

Le catalogue affiche les produits sous forme de grille responsive de cards. Chaque card présente l'image du produit, son nom, sa catégorie, son prix et son stock disponible. Trois filtres permettent d'affiner l'affichage : une barre de recherche par nom (filtre en temps réel), un sélecteur de catégorie et deux champs de prix minimum et maximum. Des skeleton loaders s'affichent pendant le chargement initial.

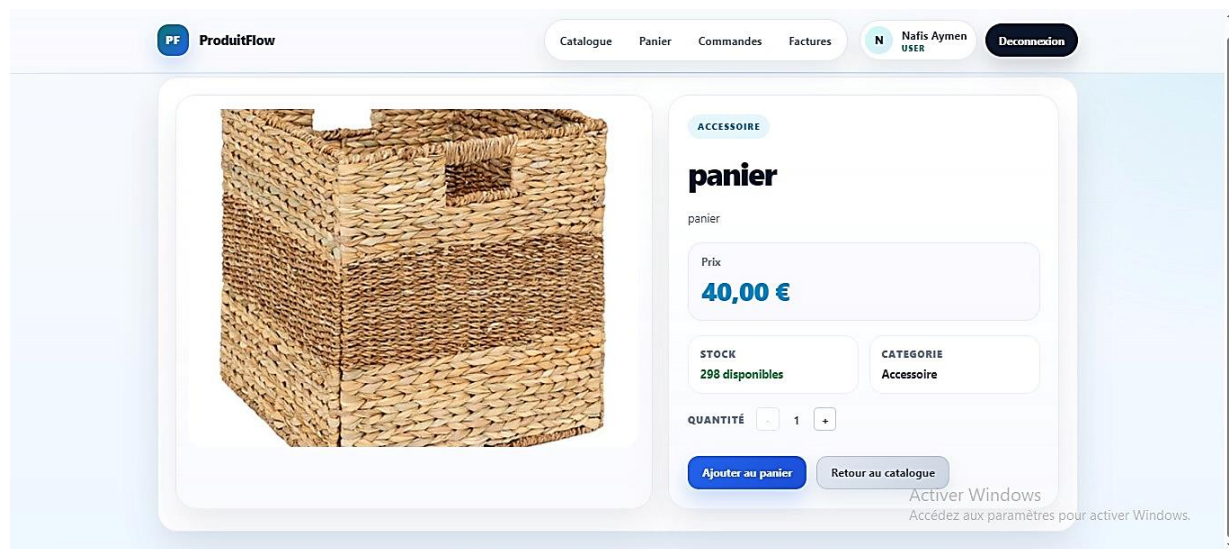


Figure 33 : Fiche Produit — sélecteur de quantité borné au stock

La fiche produit présente toutes les informations du produit sélectionné. Le sélecteur de quantité +/- est borné entre 1 et le stock disponible : le bouton - est désactivé si la quantité vaut 1, le bouton + est désactivé si la quantité atteint le stock. Le clic sur 'Ajouter au panier' appelle `addToCart(item)` du store Zustand. Si le produit est déjà présent dans le panier, la quantité est incrémentée. Un message de succès s'affiche pendant 3 secondes.

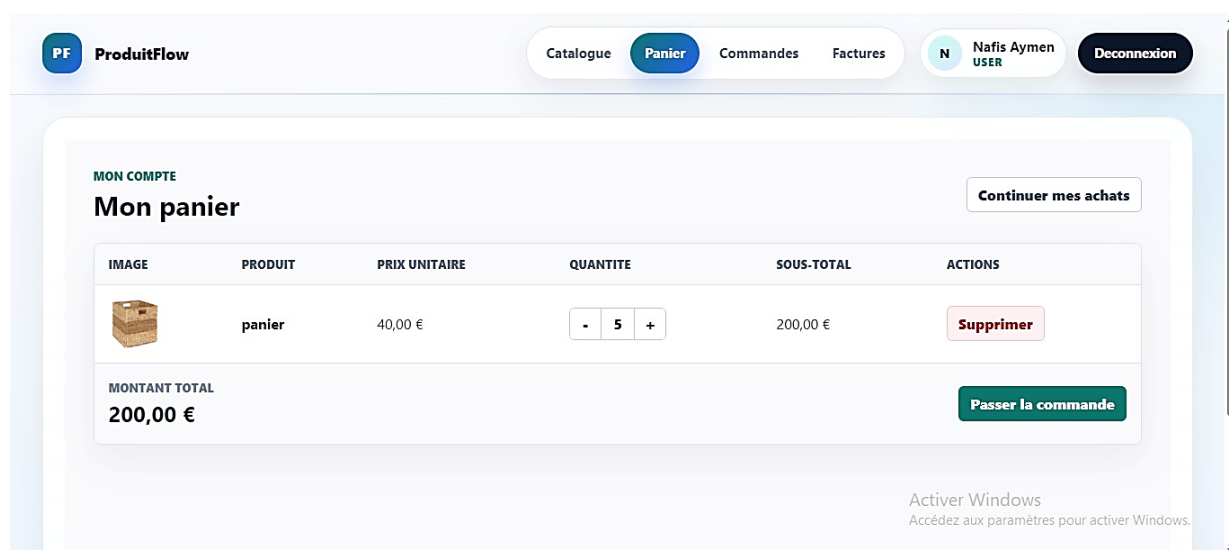


Figure 34 : Page Panier — gestion des articles et passage de commande

La page Panier lit directement depuis le store Zustand. Elle affiche un tableau avec pour chaque article : l'image, le nom, le prix unitaire, un sélecteur de quantité modifiable, le sous-total et un bouton de suppression. Le total général est recalculé dynamiquement. Le bouton 'Passer la

commande' est désactivé si le panier est vide. Après confirmation, la commande est créée en base, le panier est vidé via clearCart() et l'utilisateur est redirigé vers ses commandes.

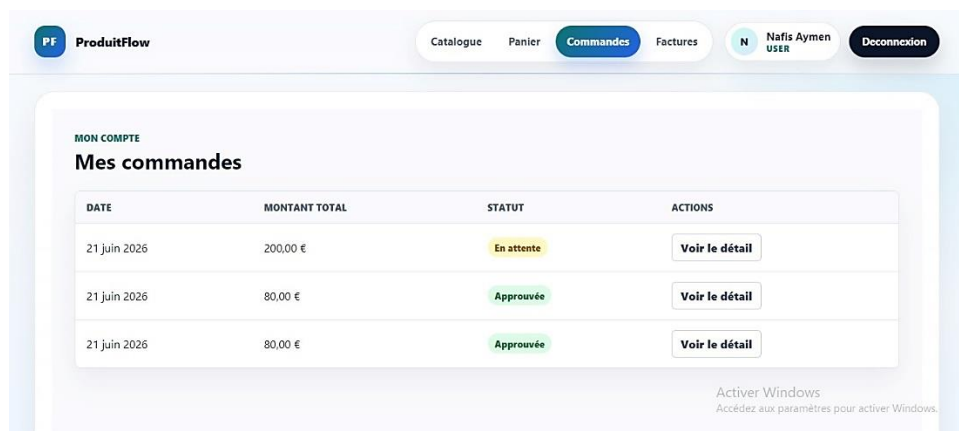


Figure 35 : Page Commandes Utilisateur — statut coloré et modal détail

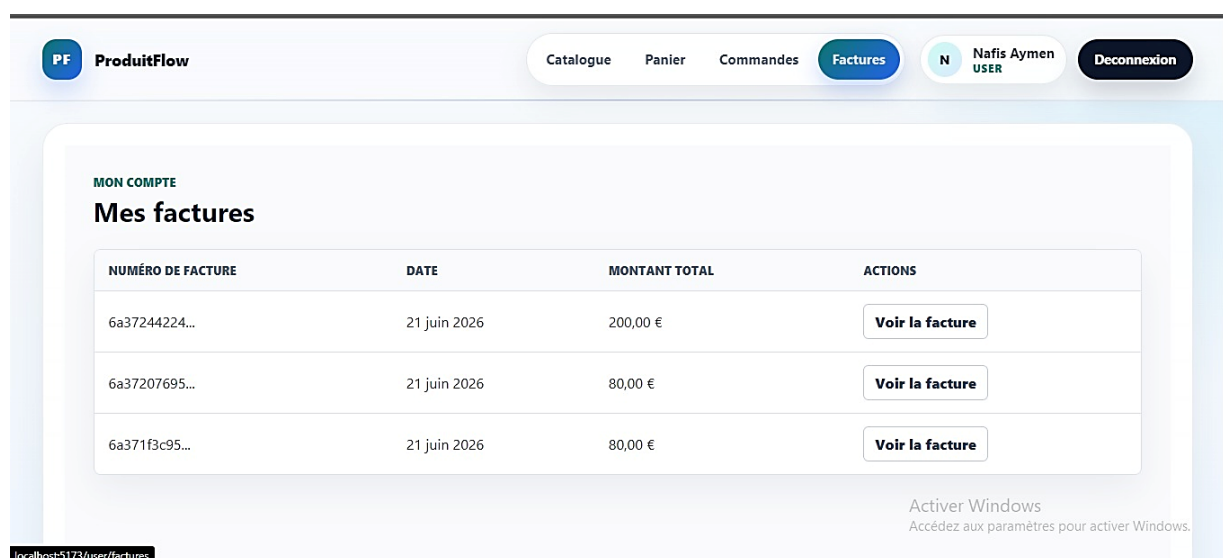


Figure 36 : Page Factures Utilisateur — liste et modal détail

5. Points Techniques Clés Implémentés

5.1. Architecture Auth avec AuthContext et useReducer

L'authentification repose sur un pattern Context + Reducer permettant une gestion d'état prévisible et testable. AuthState contient quatre propriétés : user (User | null), token (string | null), loading (boolean) et error (string | null). Le reducer gère quatre actions : LOGIN_START (passage en mode chargement), LOGIN_SUCCESS (stockage user + token), LOGIN_FAILURE (stockage de l'erreur) et LOGOUT (réinitialisation complète). Le flag isInitialized permet

d'attendre la restauration de l'état depuis sessionStorage avant d'afficher l'interface, évitant ainsi le flash de redirection indésirable au chargement.

5.2. Gestion du Panier avec Zustand

Le choix de Zustand pour la gestion du panier a été motivé par sa légèreté (aucun boilerplate Redux), sa simplicité d'utilisation (store en une fonction create()) et sa compatibilité native avec React 18. Le cartStore expose six actions : addToCart() qui incrémente la quantité si le produit est déjà présent, removeFromCart() par produitId, updateQuantity() avec validation des bornes, clearCart() après passage de commande, getTotalItems() et getTotalPrice() pour les totaux. La suppression de la dépendance à localStorage — remplacée par le store Zustand — a résolu l'incohérence découverte lors de l'intégration entre la fiche produit et la page panier.

5.3. Populate Mongoose et Helper getUserNom()

Les modèles Commande et Facture référencent userId comme Schema.Types.ObjectId avec ref: 'User', permettant l'utilisation de .populate('userId', 'nom') dans les contrôleurs admin. La transformation toJSON dans le modèle Commande.ts extrait userName depuis l'objet populé pour simplifier son utilisation côté frontend. Le helper getUserNom() côté frontend gère les deux cas de retour possibles : si userId est une chaîne de caractères, il retourne directement cette valeur ; si c'est un objet (objet populé), il retourne userId.nom.

5.4. Workflow Commande → Stock → Facture

Le contrôleur commandeController.ts implémente la logique critique de validation d'une commande en trois étapes atomiques séquentielles : (1) mise à jour du statut de la commande via Commande.findByIdAndUpdate(), (2) décrémentation du stock de chaque produit via un Promise.all() sur les lignes de commande, chacune déclenchant Produit.findByIdAndUpdate(ligne.produitId, { \$inc: { stock: -ligne.quantite } }), et (3) création d'une nouvelle Facture avec les lignes de la commande, l'userId et le montant total. Ce workflow garantit que le stock n'est décrémenté que pour les commandes effectivement approuvées, et que chaque commande approuvée génère exactement une facture.

5.5. Corrections d'Intégration

La phase d'intégration (tâche 11 du projet) a nécessité la résolution de plusieurs problèmes découverts lors du merge de la branche de Manal :

- Encodage UTF-8 corrompu : les fichiers de Manal présentaient des caractères accentués corrompus (ex: ApprouvÃ©e → Approuvée). Résolution par réenregistrement de tous les fichiers concernés en UTF-8 via VS Code.
- Routes user sans UserLayout : les pages user étaient définies en routes plates dans App.tsx sans wrapper UserLayout, rendant la navbar absente. Résolution par restructuration en routes imbriquées avec Outlet.
- Incohérence cartStore / localStorage : la page Panier lisait depuis localStorage tandis que la fiche produit écrivait dans Zustand. Résolution par unification complète vers Zustand.
- Références Mongoose incorrectes : le type de userId était String dans les modèles Commande.ts et Facture.ts, empêchant le populate. Résolution par migration vers Schema.Types.ObjectId avec ref: 'User'.
- Stock décrémenté au mauvais moment : le stock était décrémenté à la création de la commande plutôt qu'à l'approbation. Résolution par déplacement de la logique \$inc dans le handler d'approbation.
- Calcul des revenus incorrect : le dashboard comptabilisait toutes les commandes. Résolution par ajout d'un filtre .filter(c => c.statut === 'approuvee') avant le calcul du total.

6. Conclusion

La réalisation de ProduitFlow a permis de concrétiser l'ensemble des 35 besoins fonctionnels spécifiés dans un délai de 26 jours. La stack technique choisie — React 18, TypeScript, Express, MongoDB, Zustand — s'est révélée parfaitement adaptée aux exigences du projet. Les défis d'intégration rencontrés ont constitué des apprentissages techniques précieux, notamment sur la gestion de l'état global, les relations Mongoose avec populate et la discipline nécessaire dans un développement multi-branches.

Chapitre 6 : Étude Entrepreneuriale

1. Introduction

Ce chapitre examine ProduitFlow sous l'angle entrepreneurial en adoptant la démarche d'un porteur de projet souhaitant transformer cette solution académique en entreprise commerciale viable. L'objectif est de démontrer que ProduitFlow répond à un besoin réel du marché marocain, possède un potentiel commercial avéré et peut constituer la base d'un projet d'entreprise structuré et rentable à moyen terme.

2. Contexte et Identification du Problème

Le Maroc compte plus de 93 000 PME formelles selon le Haut-Commissariat au Plan (HCP), représentant 40% du PIB national et 50% des emplois du secteur privé. Or, une grande majorité de ces entreprises gèrent encore leurs processus commerciaux de manière manuelle ou via des outils inadaptés : tableurs, carnets de commandes, factures Word générées manuellement.

Ce déficit de digitalisation engendre des inefficacités opérationnelles majeures : erreurs de stock, délais de traitement des commandes, absence de traçabilité financière, et impossibilité de générer des indicateurs de performance fiables. Les solutions existantes sont soit trop complexes et coûteuses (ERP comme SAP ou Odoo), soit trop généralistes et inadaptées au B2B interne. Il existe donc un vide de marché pour des solutions SaaS légères, accessibles et spécialisées dans la gestion commerciale des PME marocaines.

3. Présentation de la Solution et Valeur Ajoutée

ProduitFlow est une solution SaaS B2B de gestion de produits, commandes et facturation, conçue spécifiquement pour les PME marocaines. Elle se distingue des solutions concurrentes par cinq avantages clés : déploiement immédiat sans installation ni formation technique, workflow de commande complet avec validation administrative et facturation automatique, gestion des stocks en temps réel avec alertes de stock faible, tableau de bord avec indicateurs financiers basés sur les commandes approuvées, et système de rôles adapté à la hiérarchie des PME.

4. Origine de l'Idée Entrepreneuriale

L'idée de ProduitFlow est née d'une observation terrain : de nombreuses PME de la région casablancaise utilisent encore Excel pour gérer leurs commandes clients et Word pour générer leurs factures. Ces processus manuels génèrent des pertes de temps considérables et des risques

d'erreurs qui impactent directement la satisfaction client et la fiabilité des données financières. L'opportunité identifiée est de proposer une alternative digitale simple, abordable et opérationnelle immédiatement.

Business Model Canvas

Tableau 10 : Business Model Canvas (BMC)

PARTENAIRES CLÉS	ACTIVITÉS CLÉS	PROPOSITION DE VALEUR	RELATIONS CLIENTS	SEGMENTS DE CLIENTÈLE
- Fournisseurs cloud : AWS / Vercel / Railway - MongoDB Atlas (base de données) - Cabinets comptables (prescripteurs) - CCG Maroc (financement Innov Invest) - OMPIC (propriété intellectuelle)	- Développement et maintenance de la plateforme - Acquisition et onboarding des clients - Support technique et relation client - Marketing de contenu et SEO - Veille technologique et évolution produit	- Digitalisation simple et immédiate de la gestion commerciale - Workflow commande → approbation → stock → facture entièrement automatisé - Ni tableur limité, ni ERP complexe — le juste milieu accessible et local	- Essai gratuit 30 jours sans carte bancaire - Support email (Starter), prioritaire (Business), dédié (Enterprise) - Programme de référencement (−10% par client recommandé) - Onboarding assisté + migration des données existantes	- PME marocaines 5–100 employés - Secteurs : commerce de gros, distribution, négoce, artisanat structuré - Région Casablanca-Settat (Phase 1) → national (Phase 2) - Responsables commerciaux sans DSI dédié
	RESSOURCES CLÉS - Application web ProduitFlow (code + infrastructure cloud) - Équipe technique (développeurs full-stack) - Base de clients et réputation (bouche-à-oreille) - Marque déposée à l'OMPIC - Infrastructure cloud scalable	- Zéro formation requise, déploiement en moins d'une heure - Tableau de bord analytique avec indicateurs financiers en temps réel	CANAUX - Site web vitrine avec démonstration interactive - Vente directe (équipe commerciale terrain) - Partenariats cabinets comptables + associations PME - LinkedIn + SEO + blog professionnel - Salons professionnels (GITEX Africa, Forum PME Casablanca)	
STRUCTURE DE COÛTS			FLUX DE REVENUS	

• Charges fixes : 33 500 MAD/mois (salaires, hébergement, marketing, comptabilité) • Charges variables : ~3 700 MAD/mois (support, cloud scalable, publicité) • Investissement initial : 98 780 MAD • Modèle cost-driven optimisé pour la scalabilité SaaS • Seuil de rentabilité : ~47 924 MAD/mois (~80 clients actifs, M18–20)	• Offre Starter : 299 MAD/mois (jusqu'à 3 utilisateurs) • Offre Business : 599 MAD/mois (jusqu'à 10 utilisateurs) • Offre Entreprise : sur devis (utilisateurs illimités) • Revenus récurrents mensuels (MRR) — encaissement avant décaissement • Période d'essai gratuite 30 jours → conversion vers abonnement payant
---	---

5. Étude de Marché

5.1. Analyse de la Demande

Notre cible principale est constituée des PME marocaines du secteur commercial et de distribution, employant entre 5 et 100 personnes, souhaitant digitaliser leur gestion commerciale sans investir dans un ERP coûteux. Ces entreprises présentent un budget IT limité mais croissant, un besoin de simplicité d'utilisation pour des équipes non techniques, une sensibilité au prix favorable au modèle d'abonnement mensuel, et une volonté croissante de traçabilité et de reporting fiable.

5.2. Analyse de la Concurrence

Tableau 11 : Analyse de la concurrence

Concurrent	Type	Points forts	Points faibles	Différenciation ProduitFlow
Odoo	ERP généraliste SaaS	Très complet, modulaire	Complexe, formation longue, 12–25 €/user/mois	Simple, déployable en heures, prix accessible
Zoho Inventory	Gestion stocks SaaS	Interface intuitive	Peu adapté au contexte marocain, 39–99 \$/mois	Local, workflow PME B2B intégré
WooCommerce	E-commerce WordPress	Flexible, open-source	Orienté B2C, pas B2B interne	Workflow commande-approbation-facture natif

Excel / Tableurs	Solution manuelle	Accessible, universel	Aucune automatisation, erreurs fréquentes	Automatisation complète stock + facture
------------------	-------------------	-----------------------	---	---

5.3. Analyse des Fournisseurs

Tableau 12 : Analyse des fournisseurs

Type de fournisseur	Exemple	Rôle	Risque de dépendance
Hébergement Cloud	AWS / Vercel / Railway	Déploiement frontend et backend	Moyen — alternatives multiples disponibles
Base de données Cloud	MongoDB Atlas	Stockage et gestion des données	Faible — solution open-source, exportable
Authentification	JWT (interne)	Sécurisation des sessions utilisateur	Nul — solution développée en interne
Registraire domaine	GoDaddy / OVH	Nom de domaine de la plateforme	Faible — marché très concurrentiel

5.4. Réglementations du Secteur

- Protection des données personnelles : conformité avec la loi 09-08 relative à la protection des données personnelles au Maroc (CNDP — Commission Nationale de contrôle de la protection des Données à caractère Personnel).
- Facturation électronique : la solution génère des factures dématérialisées, en accord avec les dispositions fiscales marocaines sur la facturation électronique (circulaire DGI 2023).
- Sécurité des données : obligation de sécuriser les données clients par des moyens techniques appropriés (hachage bcryptjs, HTTPS, tokens JWT à expiration).
- Création d'entreprise : immatriculation SARL auprès de l'OMPIC, inscription au Registre de Commerce, affiliation à la CNSS.

5.5. Outils d'Analyse Stratégique

Matrice SWOT :

Matrice SWOT — ProduitFlow

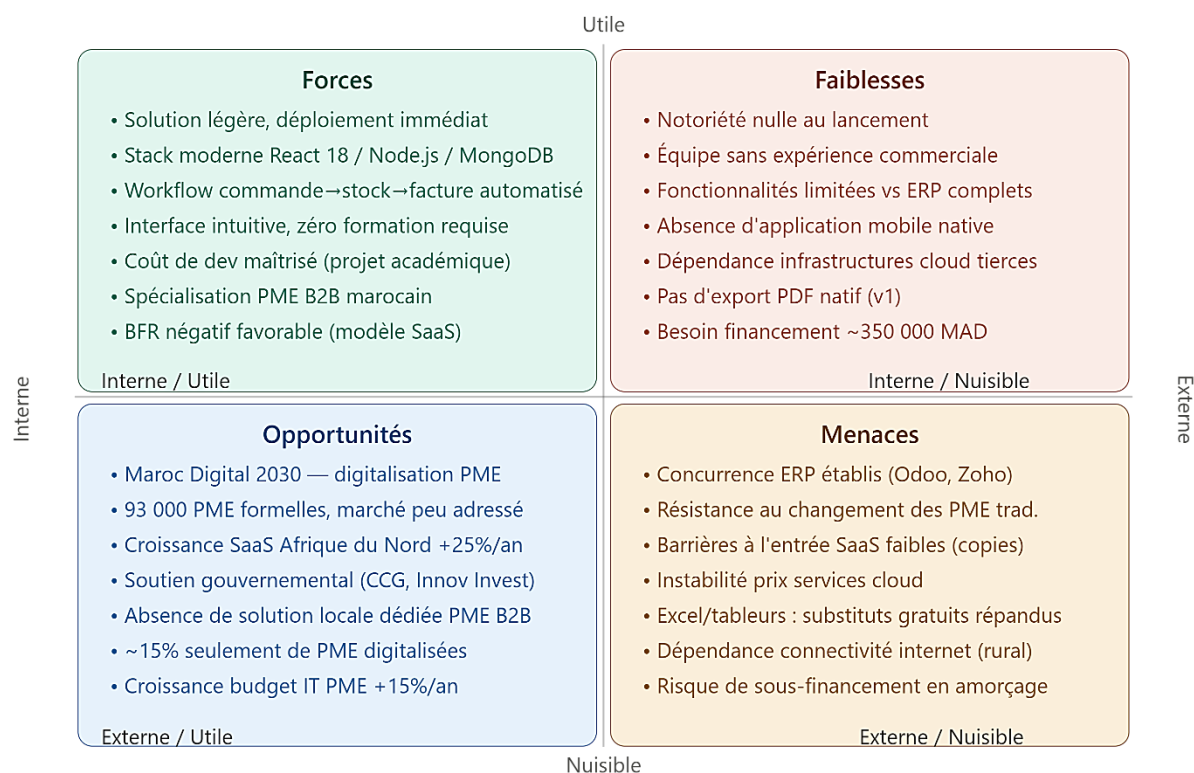


Figure 37 : Matrice SWOT de ProduitFlow

Tableau 13 : Tableau de la Matrice SWOT

Forces	Faiblesses
• Solution légère, rapide à déployer sans infrastructure • Stack moderne, maintenable et scalable • Workflow commande-stock-facture entièrement automatisé • Interface intuitive ne nécessitant aucune formation • Coût de développement initial maîtrisé (projet académique) • Spécialisation PME B2B marocain	• Notoriété nulle au lancement • Équipe fondatrice sans expérience commerciale significative • Fonctionnalités limitées par rapport aux ERP complets • Absence de version mobile native • Dépendance aux infrastructures cloud tierces
Opportunités	Menaces
• Programme Maroc Digital 2030 favorisant la digitalisation des PME • Marché marocain des PME très peu digitalisé (~15% seulement)	• Concurrence des ERP établis disposant de ressources importantes • Résistance au changement des PME traditionnelles • Risque

• Croissance du SaaS en Afrique du Nord (+25%/an) • Absence de solution locale dédiée PME B2B sur ce segment • Soutien gouvernemental aux startups (CCG, Innov Invest)	• de copie du concept (faibles barrières à l'entrée SaaS) • Instabilité potentielle des prix des services cloud • Dépendance à la connectivité internet pour les zones rurales
--	--

Analyse PESTEL :

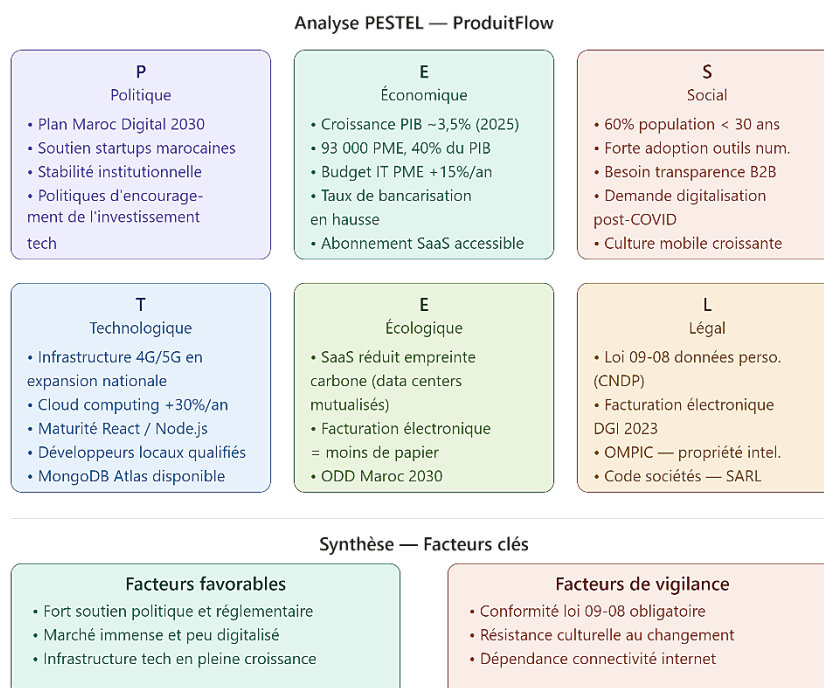


Figure 38 : Analyse PESTEL de l'environnement de ProduitFlow

Tableau 14 : Tableau d'Analyse PESTEL

Dimension	Facteurs clés d'analyse
Politique	Soutien gouvernemental à la transformation digitale (Plan Maroc Digital 2030), stabilité institutionnelle favorable aux investissements tech, politiques d'encouragement des startups marocaines.
Économique	Croissance du PIB marocain ~3,5% (2025), marché PME représentant 40% du PIB, budget IT des PME en augmentation de 15%/an, taux de bancarisation en hausse facilitant les paiements SaaS.

Social	Population jeune (60% < 30 ans) et connectée, forte adoption des outils numériques professionnels, besoin croissant de transparence dans les transactions B2B, demande de digitalisation post-COVID.
Technologique	Infrastructure 4G/5G en expansion sur tout le territoire, adoption du cloud computing en hausse de 30%/an au Maroc, maturité des frameworks JS modernes (React, Node.js), disponibilité de développeurs locaux qualifiés.
Écologique	SaaS réduisant l'empreinte carbone vs solutions on-premise (data centers mutualisés), facturation électronique réduisant la consommation de papier, conformité avec les objectifs de développement durable du Maroc.
Légal	Loi 09-08 sur la protection des données personnelles (CNDP), réglementation sur la facturation électronique (DGI), OMPIC pour la protection de la propriété intellectuelle, Code des sociétés pour la création SARL.

Les 5 Forces de Porter :

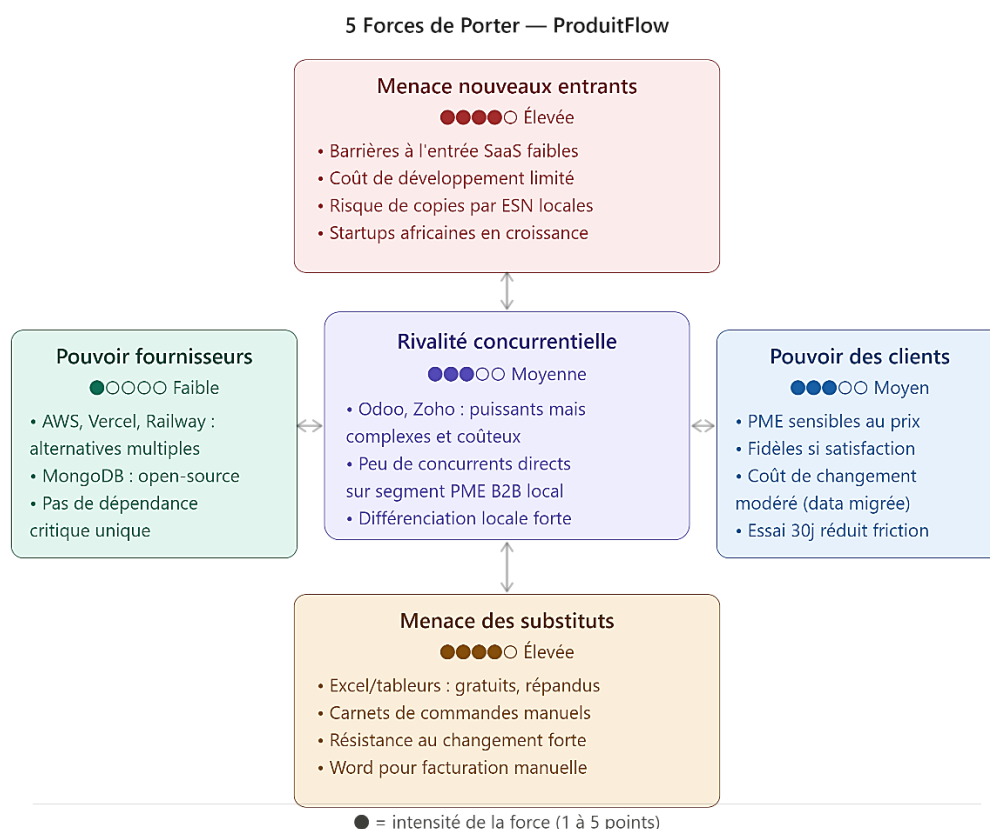


Figure 39 : Analyse des 5 Forces de Porter — ProduitFlow

Tableau 15 : Tableau des 5 Forces de Porter

Force	Intensité	Analyse détaillée
Rivalité concurrentielle	Moyenne	Concurrence d'Odoo et Zoho mais positionnement différencié sur la légèreté et la localisation marocaine. Peu de concurrents directs sur le segment PME B2B marocain.
Menace nouveaux entrants	Élevée	Barrières à l'entrée faibles dans le SaaS (coût de développement limité), risque de copies du concept par d'autres startups ou ESN marocaines.
Pouvoir de négociation fournisseurs	Faible	Alternatives multiples pour l'hébergement (AWS, Vercel, Railway, OVH), MongoDB open-source non propriétaire. Pas de dépendance critique à un seul fournisseur.
Pouvoir de négociation clients	Moyen	PME sensibles au prix mais fidèles si la solution répond à leurs besoins. Coût de changement modéré une fois les données et processus migrés vers ProduitFlow.
Menace substituts	Élevée	Excel et tableurs restent des substituts gratuits largement utilisés. La résistance au changement des PME traditionnelles constitue le principal frein à l'adoption.

6. Marketing Stratégique

6.1. Segmentation, Ciblage, Positionnement (SCP)

Segmentation : le marché est segmenté selon trois critères — la taille de l'entreprise (5–100 employés), le secteur d'activité (commerce de gros, distribution, négoce, artisanat structuré) et la maturité digitale (entreprises en phase de première digitalisation). Trois segments sont identifiés : early adopters (startups et PME jeunes), PME en croissance, et PME traditionnelles en transformation.

Ciblage : le segment prioritaire est constitué des PME casablancaises de 10 à 50 employés, en phase de première digitalisation, avec un responsable commercial identifié et sans DSI dédié. L'axe géographique de pénétration est la région Casablanca-Settat (premier marché), avec extension nationale en phase 2.

Positionnement : ProduitFlow se positionne comme la solution SaaS de gestion commerciale la plus simple et la plus accessible pour les PME marocaines — ni un tableur limité, ni un ERP complexe, mais le juste milieu intelligent et local.

6.2. Marketing Mix (4P)

- **Produit** : ProduitFlow est proposé comme une solution SaaS web complète, accessible via navigateur sans installation. Les fonctionnalités clés incluent la gestion de produits, le workflow de commande avec approbation, la facturation automatique et le tableau de bord analytique. Valeur clé : zéro formation requise, déploiement en moins d'une heure.
- **Prix** : modèle d'abonnement mensuel SaaS avec trois offres. Offre Starter à 299 MAD/mois (jusqu'à 3 utilisateurs, 500 produits, support email). Offre Business à 599 MAD/mois (jusqu'à 10 utilisateurs, produits illimités, support prioritaire). Offre Entreprise sur devis (utilisateurs illimités, fonctionnalités personnalisées, SLA garanti). Période d'essai gratuite de 30 jours sans carte bancaire.
- **Distribution** : accès direct via navigateur web, sans intermédiaire. Canal de vente directe via une équipe commerciale interne. Partenariats avec des cabinets comptables et des associations de PME marocaines. Présence sur les plateformes B2B (LinkedIn, annuaires professionnels CCI).
- **Communication** : site web vitrine avec démonstration interactive et vidéos tutoriels, LinkedIn et blog professionnel pour le marketing de contenu, SEO ciblé sur les mots-clés 'logiciel gestion PME Maroc' et 'application commandes facturation', participation aux salons professionnels (GITEX Africa, Forum PME Casablanca), programme de référencement client (réduction de 10% pour chaque client recommandé).

6.3. Stratégie d'Acquisition Clients

La stratégie d'acquisition repose sur un funnel en quatre étapes :

- **Notoriété (Awareness)** : contenu SEO, présence LinkedIn, participations aux événements tech marocains et aux rencontres des chambres de commerce.
- **Intérêt (Interest)** : démonstrations produit en ligne (webinars), études de cas clients, essai gratuit 30 jours sans engagement.
- **Décision (Decision)** : accompagnement personnalisé lors de l'onboarding, migration assistée des données existantes (import CSV), support technique réactif.

- Fidélisation (Loyalty) : programme de référencement (10% de réduction par client recommandé), mises à jour régulières, écoute des besoins et roadmap co-construite avec les premiers clients.

6.4. Stratégie de Confiance

La construction de la confiance client est essentielle dans le segment PME, où la réputation et le bouche-à-oreille jouent un rôle majeur. Notre stratégie repose sur :

- Transparence tarifaire : affichage public des tarifs, aucun frais caché, période d'essai sans carte bancaire.
- Sécurité des données : communication claire sur les mesures de sécurité (HTTPS, hachage bcryptjs, tokens JWT), conformité déclarée avec la loi 09-08.
- Témoignages et références : collecte active d'avis clients dès les premiers mois, publication d'études de cas, badges de confiance sur le site vitrine.
- Support réactif : engagement de réponse sous 24h pour l'offre Starter, sous 4h pour l'offre Business, avec un responsable dédié pour Enterprise.
- Garantie de continuité de service : SLA de disponibilité 99,5% publiés, procédures de sauvegarde régulières documentées.

7. Étude de Faisabilité Technique

La faisabilité technique de ProduitFlow en tant qu'entreprise commerciale repose sur les ressources humaines et matérielles suivantes :

Tableau 16 : Ressources techniques et humaines requises

Ressource	Quantité	Rôle	Coût estimé / mois (MAD)
Développeur full-stack senior	1	Maintenance, nouvelles fonctionnalités, architecture	8 000–12 000
Développeur junior / alternant	1–2	Support, corrections, tests, documentation	4 000–6 000
Commercial / Business Developer	1	Acquisition clients, démos, partenariats	6 000–8 000
Serveur cloud (AWS/Vercel + MongoDB Atlas)	1 plan	Hébergement production scalable	800–2 000

Domaine + SSL	1	Identité web sécurisée (produitflow.ma)	150
Outils SaaS (GitHub, Notion, Intercom)	Suite	Développement, gestion, support client	500

8. Étude de Faisabilité Financière

8.1. Investissement Initial

Tableau 17 : Investissement initial

Élément d'investissement	Quantité	Coût unitaire (MAD)	Coût total (MAD)
Développement logiciel (réalisé — PFA)	1	0 (apport en industrie)	0
Hébergement cloud (première année)	12 mois	1 000 MAD/mois	12 000
Nom de domaine + SSL (1 an)	1	800 MAD	800
Design UI/UX professionnel	1 prestation	5 000 MAD	5 000
Marketing de lancement (SEO + réseaux)	1	8 000 MAD	8 000
Création juridique SARL (greffe, statuts)	1	3 000 MAD	3 000
Matériel informatique (2 postes)	2	8 000 MAD/poste	16 000
Fonds de roulement initial (3 mois charges)	3 mois	15 000 MAD/mois	45 000
Provision imprévus (10%)	—	—	8 980

TOTAL INVESTISSEMENT INITIAL : 98 780 MAD

8.2. Prévisions de Ventes — Scénario Conservateur (Année 1)

Tableau 18 : Prévisions de ventes (scénario conservateur)

Offre	Prix (MAD/mois)	Clients M6	CA M6 (MAD)	Clients M12	CA M12 (MAD)
Starter	299	5	1 495	15	4 485

Business	599	2	1 198	8	4 792
Enterprise	1 500	0	0	2	3 000
TOTAL	—	7	2 693	25	12 277

CA annuel estimé Année 1 (scénario conservateur) : ~85 000 MAD

8.3. Charges Variables

Tableau 19 : Charges variables

Charge variable	Coût estimé / mois (MAD)	Base de calcul
Hébergement cloud (scalable au nombre de clients)	500–2 000	~80 MAD / client actif
Support client (temps agent)	1 500	10h × 150 MAD/h
Commission paiement en ligne (CMI / Stripe)	~200	2% du CA encaissé
Budget publicité digitale variable	1 000	Campagnes Google Ads / LinkedIn

8.4. Charges Fixes Mensuelles

Tableau 20 : Charges fixes mensuelles

Charge fixe	Montant mensuel (MAD)
Salaires (1 développeur senior + 1 junior)	16 000
Commercial / Business Developer	7 000
CNSS + charges sociales patronales (27%)	6 210
Hébergement serveur (plan fixe de base)	800
Abonnements logiciels (GitHub, Notion, Intercom)	500
Marketing fixe (SEO, contenu, réseaux sociaux)	1 500
Comptabilité / juridique (expert-comptable)	1 000
Autres frais généraux	490

TOTAL CHARGES FIXES : 33 500 MAD/mois

8.5. Besoin en Fonds de Roulement (BFR)

Le Besoin en Fonds de Roulement représente le besoin de financement à court terme lié au décalage entre les encaissements et les décaissements opérationnels. Dans le contexte d'un SaaS à abonnement mensuel, le BFR est structurellement faible car les clients paient en avance (abonnement mensuel prélevé en début de mois).

Tableau 21 : Calcul du Besoin en Fonds de Roulement (BFR)

Composante du BFR	Valeur (MAD)	Hypothèse
Stocks (actif)	0	SaaS — aucun stock physique
Créances clients (actif)	2 050	Délai moyen de paiement 5 jours (= CA M12 $\times$ 5/30)
Dettes fournisseurs (passif)	3 500	Délai moyen de paiement fournisseurs 30 jours
Charges sociales dues (passif)	6 210	CNSS payée le mois suivant
BFR = Actif circulant – Passif circulant	= 2 050 – (3 500 + 6 210) = -7 660 MAD	BFR négatif favorable (encaissement avant décaissement)

Le BFR est négatif (-7 660 MAD), ce qui est favorable et caractéristique des modèles SaaS à abonnement mensuel : les clients paient avant que les charges soient décaissées. Ce BFR négatif constitue un avantage de trésorerie structurel pour ProduitFlow.

8.6. Trésorerie Prévisionnelle — 12 Premiers Mois

Tableau 22 : Trésorerie prévisionnelle — 12 premiers mois

Mois	CA (MAD)	Charges totales (MAD)	Résultat mensuel (MAD)	Trésorerie cumulée (MAD)
M1	0	33 500	-33 500	-33 500
M2	599	33 700	-33 101	-66 601
M3	1 197	33 700	-32 503	-99 104
M4	1 796	33 800	-32 004	-131 108
M5	2 394	33 800	-31 406	-162 514
M6	2 693	34 000	-31 307	-193 821
M7	4 485	34 200	-29 715	-223 536
M8	6 278	34 500	-28 222	-251 758
M9	8 070	34 800	-26 730	-278 488

M10	9 863	35 000	-25 137	-303 625
M11	11 090	35 200	-24 110	-327 735
M12	12 277	35 500	-23 223	-350 958

Besoin de financement total estimé : ~350 000 MAD (fonds propres + apports associés + dispositif Innov Invest CCG)

8.7. Résultat Prévisionnel et Seuil de Rentabilité

Tableau 23 : Résultat prévisionnel et seuil de rentabilité

Élément	Valeur (MAD)
Chiffre d'affaires mensuel (M12)	12 277
Charges variables mensuelles (M12)	3 700
Marge sur coût variable (MCV)	8 577
Taux de marge sur coût variable	69,9%
Charges fixes mensuelles	33 500
Résultat mensuel (M12)	-23 223 (déficitaire)
Seuil de rentabilité (CA mensuel requis)	47 924 MAD/mois
Nombre de clients Business équivalents requis	~80 clients actifs
Point mort estimé (mois)	Mois 18–20 (scenario optimiste)

Analyse : le projet n'atteint pas la rentabilité en année 1, ce qui est cohérent avec la réalité des startups SaaS en phase d'amorçage — le modèle SaaS présente des coûts fixes élevés au démarrage compensés par des revenus récurrents croissants. La rentabilité est visée au 18^e mois avec environ 80 clients actifs. Le besoin en financement de ~350 000 MAD est mobilisable via le programme Innov Invest de la CCG (Caisse Centrale de Garantie) ou des investisseurs providentiels de l'écosystème startup marocain.

9. Aspect Juridique

La forme juridique retenue est la Société à Responsabilité Limitée (SARL), adaptée au contexte de création d'une startup par trois associés fondateurs. Ce choix est justifié par la responsabilité limitée des associés à leurs apports, le capital social minimum d'1 MAD au Maroc, la crédibilité commerciale vis-à-vis des clients PME, et la facilité de cession de parts pour une future levée de fonds.

Les démarches de création comprennent : rédaction des statuts par un notaire ou un avocat d'affaires, dépôt au greffe du Tribunal de Commerce de Casablanca, publication au Bulletin Officiel, inscription à la TVA (DGI), immatriculation CNSS, et dépôt de la marque 'ProduitFlow' à l'OMPIC pour la protection de la propriété intellectuelle.

10. Stratégie de Développement

La stratégie de développement de ProduitFlow s'articule en trois phases progressives :

Phase 1 — Amorçage (0–6 mois) : finalisation du produit et déploiement cloud, acquisition des 10 premiers clients pilotes sur Casablanca, collecte intensive des retours utilisateurs, itérations rapides sur les fonctionnalités les plus demandées, constitution de l'équipe fondatrice.

Phase 2 — Croissance (6–18 mois) : extension géographique (Rabat, Marrakech, Tanger), ajout de fonctionnalités différenciantes (rapports PDF, notifications email automatiques, API d'intégration), partenariats avec des cabinets comptables et des associations de PME, objectif 80 clients actifs et rentabilité opérationnelle.

Phase 3 — Expansion (18–36 mois) : internationalisation vers l'Afrique subsaharienne francophone (Sénégal, Côte d'Ivoire, Tunisie), développement d'une application mobile native (React Native), exploration d'une levée de fonds Série A auprès de fonds spécialisés Afrique/MENA.

11. Conclusion

Cette étude entrepreneuriale démontre que ProduitFlow répond à un besoin réel et identifié sur le marché marocain des PME. Si le projet n'atteint pas la rentabilité en année 1 — ce qui est parfaitement cohérent avec la dynamique des startups SaaS en phase d'amorçage — les perspectives de croissance sont réalistes et le modèle économique est structurellement viable à moyen terme. La combinaison d'un BFR négatif favorable, d'un modèle de revenus récurrents et d'un marché cible peu adressé constitue une base solide pour le développement d'une entreprise durable.

Conclusion Générale et Perspectives

Ce projet de fin d'année a abouti au développement de ProduitFlow, une plateforme web full-stack de gestion de produits, commandes et facturation, répondant aux besoins concrets des PME souhaitant digitaliser leur chaîne commerciale. Ce rapport a couvert l'intégralité du cycle de développement, de l'analyse des besoins à la réalisation technique, en passant par la gestion de projet et l'étude entrepreneuriale.

Sur le plan technique, nous avons conçu et implémenté une solution complète reposant sur une stack moderne et cohérente (React 18 + TypeScript, Node.js + Express, MongoDB, Zustand, JWT). Les 35 besoins fonctionnels identifiés ont été entièrement réalisés. Les défis d'intégration rencontrés — corruption UTF-8, unification du state management, gestion des populations Mongoose, synchronisation du stock — ont été résolus avec rigueur et ont constitué des apprentissages techniques essentiels pour notre formation d'ingénieur.

Sur le plan de la gestion de projet, le workflow Git structuré en branches feature avec intégration contrôlée et validée a permis un développement parallèle efficace entre les trois membres de l'équipe dans un délai de 26 jours. Le réseau PERT a mis en évidence le chemin critique et permis d'anticiper les dépendances bloquantes.

Sur le plan entrepreneurial, l'étude de marché a confirmé l'existence d'un vide commercial pour des solutions SaaS légères dédiées aux PME marocaines. Le modèle d'abonnement mensuel, le BFR négatif favorable et la valeur ajoutée de l'automatisation (stock + facturation) constituent une base solide pour un développement commercial envisageable à l'issue de notre formation.

Plusieurs axes d'amélioration et d'extension sont envisagés pour les versions futures de ProduitFlow :

- Développement d'une application mobile native (React Native) pour la gestion des commandes et la consultation du catalogue en mobilité.
- Ajout d'un module de reporting avancé avec export PDF des factures et des rapports de ventes mensuels.
- Intégration d'un système de notifications par email (approbation de commande, stock faible, génération de facture).
- Déploiement cloud avec pipeline CI/CD automatisé (GitHub Actions + Vercel/Railway) pour la mise en production continue.

- Implémentation d'une API publique pour l'intégration avec des systèmes tiers (ERP, comptabilité, CRM).
- Extension géographique de la solution vers d'autres marchés africains francophones.

ProduitFlow illustre comment un projet académique rigoureux, mené avec une méthodologie structurée et une vision entrepreneuriale claire, peut constituer le point de départ d'une solution commerciale viable et scalable. Cette expérience nous a permis de mobiliser l'ensemble des compétences acquises tout au long de notre formation à l'EMSI et d'appréhender concrètement les défis du développement logiciel collaboratif en conditions réelles.

Bibliographie et Webographie

Documentation Officielle

- React Documentation — react.dev
- TypeScript Documentation — typescriptlang.org/docs
- Vite Documentation — vitejs.dev
- React Router DOM Documentation — reactrouter.com
- Zustand Documentation — github.com/pmndrs/zustand
- Axios Documentation — axios-http.com/docs
- Express.js Documentation — expressjs.com
- Mongoose Documentation — mongoosejs.com
- JSON Web Token (JWT) — jwt.io
- bcryptjs — github.com/dcodeIO/bcrypt.js

Ouvrages de Référence

- BANKS, Alex & Eve PORCELLO. Learning React. O'Reilly Media, 2020.
- FLANAGAN, David. JavaScript: The Definitive Guide. O'Reilly Media, 2020.
- FOWLER, Martin. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.

Ressources en Ligne

- MDN Web Docs — developer.mozilla.org
- Stack Overflow — stackoverflow.com
- GitHub du projet ProduitFlow — github.com/Aymen21-n/ProductFlow
- HCP Maroc (données PME) — hcp.ma
- OMPIC (propriété intellectuelle) — ompic.ma
- CCG Maroc (financement startups) — cgc.ma
- CNDP Maroc (protection données) — cndp.ma